

Private University
Horizon School of Digital
Technologies

Republic of Tunisia
Ministry of Higher Education
and Scientific Research



End Of Studies Project Report

Presented in order to obtain the National **Bachelor's Degree** in

Computer Science

Major: **Software Engineering and Information Systems**

**VisionTest-AI: An Intelligent Agent for Autonomous
Functional Testing using NLP and Computer Vision**

By: **Nour El Karia**

Defended on / / 2026 in front of the jury composed of:

Mr./Mme :

President

Mr./Mme :

Reviewer

Mme Noura Aboudi

Academic Supervisor

M. Baha Rahmouni

Professional Supervisor



Réf :

Année Universitaire : 2025 – 2026

Dedication

I dedicate this work...

*To **God Almighty**, for His infinite grace and for lighting my path.*

*To my **Parents**, my first teachers and greatest supporters. Your love and sacrifices are the foundation of my success.*

*To my **Sisters and Brother**, for your encouragement and for always believing in me.*

To my dear friends Thank you for the shared journey, the motivation, and the unforgettable memories.

*To the **Horizon School of Digital Technologies**, for the academic excellence that shaped my future.*

Acknowledgments

First and foremost, I wish to express my gratitude to the Almighty **God** for providing me with the strength and perseverance required to complete this work.

I would like to acknowledge my professional supervisors at **ADDINN Group**, **M. Mohamed Essid** and **M. Baha Rahmouni**, for their presence and for the opportunity to conduct my final year project within the company's Data/AI department.

I am especially honored to express my deepest and most sincere gratitude to my academic supervisor, **Mme Noura Aboudi**. I am profoundly grateful for her invaluable guidance, her constant availability, and the scientific rigor she provided throughout this journey. Her unwavering support and encouragement were essential in overcoming the technical challenges of this project and ensuring its academic success.

I also wish to extend my appreciation to the **Members of the Jury** for their time and effort in evaluating this work, as well as to the staff of the **Horizon School of Digital Technologies** for the education I have received over the years.

Finally, I thank all my colleagues at **ADDINN Group** for the professional environment provided during my internship.

Nour El Karia

Abstract

Functional test automation is an important part of modern software quality assurance, especially in Agile and DevOps environments where applications change frequently. However, automated tests often remain fragile because they depend on technical selectors, DOM structure, and manually written scripts. A small change in the user interface can break a test even when the business behavior of the application is still correct.

This end-of-studies project, carried out within **ADDINN Group**, presents **VisionTest-AI**, an intelligent functional testing agent designed to reduce this fragility. The system starts from business-readable Gherkin scenarios and transforms them into executable browser actions. It combines Natural Language Processing for scenario parsing and intent extraction, semantic mapping for action generation, Playwright for browser automation, YOLOv8 for UI element detection, OCR for visible text extraction, and a reporting module for JSON and HTML execution reports.

The proposed solution follows a hybrid execution strategy. First, it tries to interact with the application using generated Playwright selectors. If the selector-based execution fails, the system activates a visual fallback mechanism based on screenshot analysis, object detection, OCR, and coordinate-based interaction. This approach brings the automation process closer to the behavior of a human tester who reads a scenario, observes the interface, and interacts with visible elements.

The final prototype includes a FastAPI backend, a Streamlit user interface, NLP and semantic mapping services, a vision service, an execution service, and a reporting service. The project also highlights practical limitations, especially the need for a larger annotated dataset, better OCR robustness, and a more scalable deployment architecture. Despite these limitations, **VisionTest-AI** demonstrates the feasibility of combining semantic understanding, browser automation, and visual perception to support more resilient functional testing.

Keywords: Functional Testing, Artificial Intelligence, Gherkin, NLP, Playwright, YOLOv8, OCR, Computer Vision, Test Automation.

Contents

Dedication	i
Acknowledgments	ii
Abstract	iii
List of Figures	ix
List of Tables	x
General Introduction	1
1 General Context, Requirements and Project Overview	3
1.1 Introduction	4
Chapter Overview	4
1.2 Host Organization and Project Environment	4
1.3 Project Context and Problem Statement	5
1.4 Existing Solutions and Positioning	5
1.4.1 Traditional Automation Tools	5
1.4.2 AI-Based Testing Platforms	6
1.4.3 Positioning of VisionTest-AI	6
1.5 Proposed Solution	6
1.6 Requirements Analysis	9
1.6.1 Functional Requirements	9
1.6.2 Non-Functional Requirements	9
1.6.3 Constraints and Assumptions	9
1.7 Project Objectives	9
1.8 Project Management Methodology	10
1.9 Conclusion	10
2 Literature Review: Data and Artificial Intelligence Foundations	11
2.1 Introduction	12
Chapter Overview	12
2.2 Data Science and Artificial Intelligence	12
2.3 Machine Learning	12

2.4	Deep Learning	13
2.5	Natural Language Processing	13
2.5.1	Rule-Based NLP	13
2.5.2	Transformer-Based NLP	13
2.5.3	Hybrid NLP Strategy	14
2.6	Computer Vision	14
2.7	Object Detection and YOLO	14
2.8	OCR and Text Extraction	15
2.9	Behavior-Driven Development and Gherkin	15
2.10	Browser Automation	15
2.11	Hybrid Intelligent Testing	15
2.12	Conclusion	16
3	Semantic Intelligence: NLP and Mapping Module	17
3.1	Introduction	19
	Chapter Overview	19
3.2	Objectives of the NLP Module	19
3.3	Gherkin Parsing	20
3.3.1	Parsed Elements	20
3.3.2	Handling And and But	20
3.4	Intent Classification	21
3.4.1	Rule-Based First Version	21
3.4.2	DistilBERT-Based Improvement	21
3.4.3	Confidence Management	21
3.5	Entity Extraction	21
3.5.1	Target and Value Extraction	22
3.5.2	Regex Fallback	22
3.6	Semantic Mapping	22
3.6.1	Selector Candidate Generation	23
3.6.2	Preparing Vision Fallback	23
3.7	API Endpoints of the NLP Module	23
3.7.1	<code>/api/ia/parse-gherkin</code>	23
3.7.2	<code>/api/ia/semantic-mapping</code>	25
3.7.3	<code>/api/ia/generate-test</code>	27
3.8	Generated Playwright Script	28
3.9	Validation and Results	29
3.9.1	Observed Results	29
3.9.2	Error Analysis	30
3.10	Limitations and Possible Improvements	30
3.11	Conclusion	30

4	Visual Perception and Self-Healing Execution	31
4.1	Introduction	33
	Chapter Overview	33
4.2	Objective of the Vision Module	33
4.3	Dataset Preparation: From Failed Attempts to DOM-Assisted Labeling . . .	34
4.3.1	The Challenge of Building a UI Dataset	34
4.3.2	First Attempt: Searching for Ready-Made Datasets	34
4.3.3	Second Attempt: Playwright Screenshot Generation	35
4.3.4	Third Attempt: Manual Annotation with Roboflow and Labelling . .	35
4.3.5	Limitations of Classic Auto-Labeling	36
4.3.6	Final Solution: DOM-Assisted Auto-Labeling	37
4.3.7	UI Element Classes	38
4.3.8	Dataset Split and Preparation for Training	39
4.4	YOLOv8n Training	39
4.4.1	Why YOLOv8n	39
4.4.2	Training Configuration	40
4.4.3	Loss Function Analysis	40
4.4.4	Training Metrics	41
4.4.5	Model Evaluation	41
4.5	OCR Pipeline	42
4.5.1	Role of OCR in the Vision Module	42
4.5.2	OpenCV Preprocessing	42
4.5.3	Tesseract OCR	43
4.5.4	Text Cleaning and Matching	43
4.6	Vision API Implementation	43
4.6.1	/api/ia/analyze-screenshot	43
4.6.2	Screenshot Input	44
4.6.3	Bounding Box Detection	44
4.6.4	OCR Enrichment	44
4.6.5	Returned JSON Structure	45
4.7	Self-Healing Execution	45
4.7.1	Plan A: Playwright Selector Execution	45
4.7.2	Plan B: YOLO + OCR + Coordinate-Based Action	46
4.7.3	Retry Mechanism	46
4.7.4	Popup and Dialog Handling	46
4.8	Results and Observations	47
4.8.1	Successful Detection Cases	47
4.8.2	Successful Self-Healing Cases	47
4.8.3	Failed Cases	48
4.8.4	Error Analysis	48
4.9	Conclusion	49

5	Integration, Reporting, Deployment and Final Results	50
5.1	Introduction	52
	Chapter Overview	52
5.2	Complete Pipeline Integration	52
5.2.1	User Submits a Gherkin Scenario	52
5.2.2	NLP Parsing and Semantic Mapping	53
5.2.3	Playwright Execution	53
5.2.4	Vision Fallback When Selectors Fail	53
5.2.5	Report Generation	53
5.3	Backend Implementation	54
5.3.1	FastAPI Routers	54
5.3.2	Services Layer	55
5.3.3	Schemas and Validation	55
5.3.4	Error Handling and Logs	55
5.4	Reporting System	55
5.4.1	JSON Report Generation	56
5.4.2	HTML Report Generation	56
5.4.3	Screenshot Evidence	57
5.4.4	Report API Endpoints	57
5.4.5	Streamlit Report Viewer	58
5.5	User Interface	58
5.5.1	Scenario Studio	58
5.5.2	Script Generation View	59
5.5.3	Screenshot Analyzer	60
5.5.4	Reports Page	60
5.6	Deployment Attempts	61
5.6.1	Local Deployment	61
5.6.2	Docker Deployment	61
5.6.3	First Streamlit Cloud Attempt	62
5.6.4	Resource Limit Issue	62
5.6.5	Final Proposed Architecture: Railway Backend and Streamlit UI	62
5.6.6	CI/CD Integration Strategy	63
5.7	Testing and Validation	64
5.7.1	Unit Tests	64
5.7.2	API Tests	64
5.7.3	End-to-End Test Scenarios	64
5.7.4	Unseen-Site Benchmark	64
5.8	Actual Results	65
5.8.1	Generated JSON Reports	65
5.8.2	Generated HTML Reports	66
5.8.3	Successful Execution Examples	66
5.8.4	Failed Execution Examples	66

5.8.5	Plan A vs Plan B Usage	67
5.8.6	Performance Observations	67
5.9	Limitations	68
5.9.1	Model Accuracy Limitations	68
5.9.2	OCR Limitations	68
5.9.3	Deployment Resource Limitations	68
5.9.4	Lack of PostgreSQL and Redis Integration	68
5.10	Perspectives	68
5.10.1	Improve NLP Multilingual Support	68
5.10.2	Expand the Annotated Dataset	69
5.10.3	Add PostgreSQL and Redis	69
5.10.4	Add Authentication and Project History	69
5.10.5	Improve Cloud Deployment	69
5.11	Conclusion	69
	General Conclusion and Perspectives	70
	Bibliography	72

List of Figures

1.1	Official logo of ADDINN Group	4
1.2	Global architecture of VisionTest-AI	8
1.3	Agile Scrum lifecycle	10
3.1	NLP and semantic mapping pipeline of VisionTest-AI	20
3.2	Example request of the <code>/api/ia/parse-gherkin</code> endpoint	24
3.3	Example response of the <code>/api/ia/parse-gherkin</code> endpoint	25
3.4	Example request of the <code>/api/ia/semantic-mapping</code> endpoint	26
3.5	Example request of the <code>/api/ia/generate-test</code> endpoint	27
4.1	Manual annotation attempt using Roboflow	36
4.2	Example of incorrect or incomplete auto-labeling result	37
4.3	DOM-assisted labeling approach used to generate UI annotations	38
4.4	YOLOv8n training metrics: loss curves, precision, recall, and mAP evolution	41
4.5	Confusion matrix of the trained YOLOv8n model	42
4.6	Example request of the <code>/api/ia/analyze-screenshot</code> endpoint	43
4.7	Screenshot captured during a fallback execution	46
4.8	Example of successful UI element detection	47
5.1	Complete execution pipeline of VisionTest-AI	54
5.2	Example of an HTML execution report generated by VisionTest-AI	57
5.3	Scenario Studio interface for submitting Gherkin scenarios	59
5.4	Generated Playwright script displayed in the Streamlit interface	60
5.5	Reports page in the Streamlit interface	61
5.6	Final proposed deployment architecture	63
5.7	Example of a generated JSON report	65
5.8	Example of a generated HTML report	66

List of Tables

1.1	Comparative study of existing testing solutions	6
1.2	Project sprint planning	10
3.1	Example of parsed Gherkin steps	20
3.2	Supported intents in the NLP module	21
3.3	Main entities extracted from Gherkin steps	22
3.4	Mapping between semantic intents and Playwright methods	22
3.5	Validation examples for the NLP and semantic mapping module	29
3.6	Observed mapping results	29
3.7	NLP limitations observed during validation	30
4.1	UI classes used in the vision dataset	39
4.2	Dataset split used for YOLOv8n training	39
4.3	YOLOv8 variants and project positioning	40
4.4	YOLOv8n training configuration	40
4.5	Examples of successful self-healing situations	48
4.6	Observed failed cases in the vision module	48
5.1	Main FastAPI routers of VisionTest-AI	54
5.2	Reporting API endpoints	58
5.3	Examples of failed execution cases	67
5.4	Comparison between Plan A and Plan B	67

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BDD	Behavior-Driven Development
CNN	Convolutional Neural Network
DOM	Document Object Model
GNN	Graph Neural Network
Gherkin	Business Readable DSL for Behavior Descriptions
JSON	JavaScript Object Notation
NLP	Natural Language Processing
OCR	Optical Character Recognition
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SFE	Students for Earth
UI	User Interface
UML	Unified Modeling Language
YOLO	You Only Look Once

General Introduction

Software systems are now delivered in short cycles. New features, design changes, and bug fixes are released frequently, especially in Agile and DevOps environments. This acceleration improves productivity, but it also increases the risk of regression. For this reason, functional testing is no longer a secondary activity. It is a necessary part of software quality because it verifies whether the application works correctly from the user's point of view.

Traditional automated testing tools are powerful, but they often depend on technical selectors and page structure. This creates a maintenance problem. If a button changes its ID, if a CSS class is renamed, or if the DOM structure is refactored, the test can fail even when the application behavior is still correct. These failures reduce confidence in automation and create extra work for QA teams.

Artificial Intelligence offers new possibilities for improving this process. Natural Language Processing can help transform readable test scenarios into structured actions. Computer Vision can help detect visible interface elements from screenshots. OCR can extract text displayed on the page. Browser automation can execute the actions, and reporting can keep execution evidence for analysis.

This project, entitled **VisionTest-AI**, follows this direction. It proposes an intelligent functional testing agent that starts from Gherkin scenarios and generates executable actions. The agent first uses Playwright selectors to execute the test. If the selector strategy fails, it activates a visual fallback based on YOLOv8 and OCR. The final result is documented through JSON reports, HTML reports, screenshots, and execution metadata.

The main objectives of the project are:

- transform business-readable Gherkin scenarios into structured actions [4];
- map natural language steps to Playwright-compatible actions;
- execute functional tests automatically in a browser;
- add visual perception using YOLOv8 and OCR [27, 24];
- generate JSON and HTML reports that explain the result of each execution;
- study the limits of the prototype and propose realistic improvements.

The report is organized into five chapters:

1. **Chapter 1: General Context, Requirements and Project Overview** presents the host organization, the project environment, the problem statement, existing testing solutions, the proposed solution, the main requirements, the global architecture, and the project methodology. This chapter merges the project context with the preparation phase in order to keep the report concise and avoid repetition.
2. **Chapter 2: Literature Review: Data and Artificial Intelligence Foundations** introduces the scientific and technical background needed to understand the project. It explains Data Science, Machine Learning, Deep Learning, NLP, Computer Vision, object detection, OCR, BDD, Gherkin, and browser automation. This chapter provides the missing literature foundation requested by the supervisor.
3. **Chapter 3: Semantic Intelligence: NLP and Mapping Module** describes how the system understands Gherkin scenarios. It presents parsing, intent classification, entity extraction, semantic mapping, selector candidate generation, API endpoints, generated JSON outputs, and Playwright script generation.
4. **Chapter 4: Visual Perception and Self-Healing Execution** presents the vision layer of the project. It describes dataset preparation, the difficulties encountered during annotation, YOLOv8n training, OCR processing, the vision API, and the Plan A/Plan B self-healing execution strategy.
5. **Chapter 5: Integration, Reporting, Deployment and Final Results** explains how all modules are connected in the final prototype. It presents the backend implementation, reporting system, Streamlit interface, deployment attempts, CI/CD perspective, validation results, limitations, and future improvements.

The report ends with a general conclusion that summarizes the work, the main achievements, the difficulties encountered, and the possible perspectives for industrialization.

1 | General Context, Requirements and Project Overview

Contents

1.1	Introduction	4
	Chapter Overview	4
1.2	Host Organization and Project Environment	4
1.3	Project Context and Problem Statement	5
1.4	Existing Solutions and Positioning	5
1.4.1	Traditional Automation Tools	5
1.4.2	AI-Based Testing Platforms	6
1.4.3	Positioning of VisionTest-AI	6
1.5	Proposed Solution	6
1.6	Requirements Analysis	9
1.6.1	Functional Requirements	9
1.6.2	Non-Functional Requirements	9
1.6.3	Constraints and Assumptions	9
1.7	Project Objectives	9
1.8	Project Management Methodology	10
1.9	Conclusion	10

1.1 Introduction

Software teams are asked to deliver new features faster while keeping applications stable. This pressure makes functional testing an essential activity in modern software engineering. Functional tests verify that the application behaves correctly from the user’s point of view: opening a page, filling a form, clicking a button, or checking that an expected message appears.

In practice, automated functional tests are often fragile. Many tools depend on technical selectors such as IDs, CSS classes, XPath expressions, or DOM structure. When the interface changes, a test can fail even if the application still works correctly for the user. The **VisionTest-AI** project was proposed to reduce this fragility by combining Gherkin parsing, semantic mapping, Playwright execution, computer vision, OCR, and reporting.

Chapter Overview

This chapter presents the general context of the project. It introduces the host organization, explains the problem statement, reviews existing testing solutions, defines the objectives and requirements, and summarizes the global architecture and project methodology.

1.2 Host Organization and Project Environment

The project was carried out within **ADDINN Group**, a technology company active in digital engineering, data-oriented solutions, artificial intelligence, cybersecurity, cloud services, and business process automation. This environment was suitable for the project because **VisionTest-AI** is not only an AI experiment; it is an engineering prototype intended to answer a concrete software quality problem.

Figure 1.1 presents the official logo of the host organization where the project was carried out.



Figure 1.1: Official logo of ADDINN Group

The internship took place in a Data and Artificial Intelligence oriented context. The work required knowledge from several domains: functional testing, Behavior-Driven Development, natural language processing, browser automation, computer vision, OCR, backend development, and report generation. Professional supervision helped align the project with practical needs, while academic supervision ensured that the work respected the expected structure of an end-of-studies project.

1.3 Project Context and Problem Statement

Web applications have become more dynamic and more complex. They often include asynchronous loading, responsive layouts, popups, dynamic components, and frequent user interface updates. Manual validation of these applications is slow and repetitive, which explains why companies rely on automated testing tools such as Selenium, Playwright, and Cypress.

These tools are useful, but they do not remove all maintenance problems. A test script usually needs selectors to locate elements. If an element ID changes, if the DOM structure is refactored, or if a button is moved into another component, the test may fail for technical reasons rather than for a real functional defect.

The main problems addressed by this project are:

- **High maintenance cost:** QA engineers must frequently repair scripts after minor UI changes.
- **False failures:** A test can fail because a selector changed, not because the application behavior is wrong.
- **Limited accessibility:** Business-readable scenarios are not directly executable by automation tools.
- **Weak visual understanding:** Traditional automation reads the DOM but does not perceive the screen like a human tester.
- **Dynamic interface difficulty:** Popups, changing layouts, and visually similar components are difficult to automate reliably.

The central question can therefore be formulated as follows:

How can we design an intelligent functional testing agent capable of understanding Gherkin scenarios, generating executable actions, interacting with web interfaces, and adapting when classical selectors fail?

1.4 Existing Solutions and Positioning

1.4.1 Traditional Automation Tools

Selenium [21], **Playwright** [13], and **Cypress** [5] are widely used for web automation. They allow testers to open pages, click elements, fill fields, and verify results. These tools remain important because they provide the execution layer of many test suites. However, they still require scripts and selectors, and they do not fully solve the problem of fragile tests.

1.4.2 AI-Based Testing Platforms

Several commercial platforms try to improve automated testing using artificial intelligence. **Mabl** [11] focuses on low-code testing and self-healing. **QA Wolf** [16] provides managed Playwright-based test automation. **Applitools** [1] is known for visual AI and screenshot comparison. **Functionize** [9] uses ML and NLP for autonomous testing. **Testim** [25] improves selector stability using smart locators.

Table 1.1 summarizes the main strengths and limits of the existing tools compared with the proposed prototype.

Table 1.1: Comparative study of existing testing solutions

Solution	Main Strength	Main Limitation
Selenium / Cypress / Playwright	Strong browser automation and large ecosystem.	Requires technical scripts and selectors.
Mabl	Low-code testing and self-healing support.	Commercial cloud platform with limited internal control.
QA Wolf	Fast Playwright-based test creation.	Managed service model, less suited to internal AI experimentation.
Applitools	Strong visual regression detection.	Focused mainly on visual validation.
Functionize	Natural language support and adaptive execution.	SaaS dependency and limited transparency.
Testim	More stable smart locators.	Still strongly related to DOM attributes.
VisionTest-AI	Hybrid NLP, Playwright, YOLO/OCR fallback, and reporting.	Prototype stage; depends on dataset and OCR quality.

1.4.3 Positioning of VisionTest-AI

VisionTest-AI is positioned as a custom intelligent testing prototype. It combines semantic understanding and visual perception instead of relying only on DOM selectors. The goal is not to replace Playwright, but to enrich it with NLP-based test generation, visual fallback, and execution reporting.

1.5 Proposed Solution

The proposed solution follows a hybrid strategy. The user writes a Gherkin scenario. The NLP module parses the scenario and extracts steps, intents, targets, and values. The semantic mapping module transforms these interpreted steps into Playwright-compatible actions. The execution service first tries to perform the action with generated selectors. If this fails,

the vision service analyzes a screenshot using YOLOv8 and OCR to find the target visually. At the end, the reporting service generates JSON and HTML reports.

Figure 1.2 represents the global architecture of **VisionTest-AI**. The diagram was created by the author using Mermaid [12].

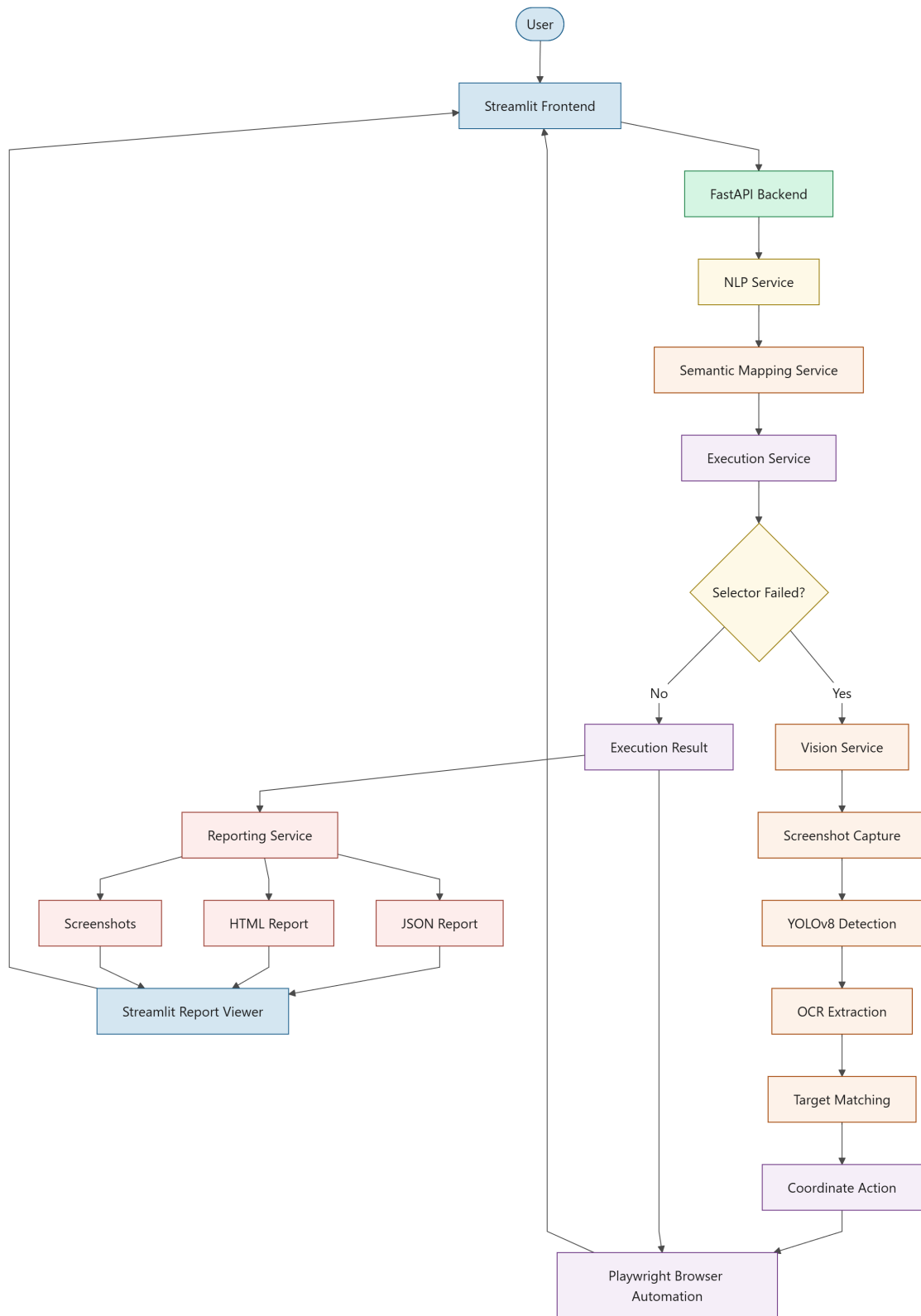


Figure 1.2: Global architecture of VisionTest-AI

The solution is organized around the following modules:

- **Frontend:** Streamlit interface for scenario submission, script generation, screenshot analysis, and report viewing.

- **Backend:** FastAPI service exposing the main API endpoints.
- **NLP service:** Gherkin parsing, intent classification, and entity extraction.
- **Semantic mapping service:** Conversion from interpreted steps to executable actions and selector candidates.
- **Execution service:** Playwright-based browser automation.
- **Vision service:** YOLOv8 and OCR-based fallback.
- **Reporting service:** JSON and HTML report generation.

1.6 Requirements Analysis

1.6.1 Functional Requirements

The system must allow the user to submit a Gherkin scenario, parse it, classify the intent of each step, extract targets and values, generate Playwright actions, execute the test, activate visual fallback when selectors fail, capture screenshots, and generate JSON and HTML reports.

1.6.2 Non-Functional Requirements

The system must be modular, maintainable, usable, extensible, and traceable. Each service should have a clear responsibility. Errors must be captured and explained in the final report instead of being hidden from the user. The prototype should also remain open for future improvements such as authentication, database storage, and background task queues.

1.6.3 Constraints and Assumptions

The current version is a prototype. Its visual fallback depends on the quality of the dataset, the accuracy of YOLO detections, and the readability of OCR output. Some dynamic websites may behave differently because of popups, network speed, or anti-bot protections. The current version also does not include full PostgreSQL, Redis, authentication, or project history integration.

1.7 Project Objectives

The functional objectives are to make Gherkin scenarios executable, reduce manual script writing, execute tests automatically, use visual fallback when needed, and generate readable execution reports. The technical objectives are to build a modular FastAPI backend [8], use Pydantic schemas [15], automate browsers with Playwright [13], integrate YOLOv8 [27] and OCR, and prepare the application for future deployment improvements.

1.8 Project Management Methodology

The project followed an iterative methodology inspired by Agile Scrum. This choice was suitable because the project included several uncertain components: NLP interpretation, visual detection, OCR quality, browser execution, and deployment constraints.

Figure 1.3 illustrates the general Scrum lifecycle used as inspiration for the project organization.

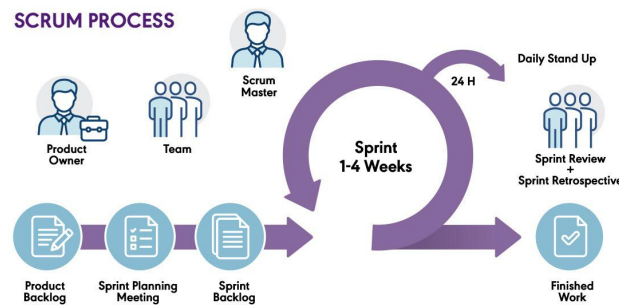


Figure 1.3: Agile Scrum lifecycle

Table 1.2 presents the sprint planning followed during the implementation of the prototype.

Table 1.2: Project sprint planning

Sprint	Main Objective	Deliverable
Sprint 1	Problem study, requirements, and first local prototype.	Initial pipeline and backlog.
Sprint 2	NLP parsing and semantic mapping.	Structured JSON and action mapping.
Sprint 3	Dataset preparation and vision model training.	Annotated dataset and YOLO model.
Sprint 4	Execution, fallback, UI, and reporting integration.	Complete prototype.

Trello [2] was used for task organization, Git [10] for version control, and VS Code [14] as the main development environment.

1.9 Conclusion

This chapter presented the general context of the project, the host organization, the testing automation problem, existing solutions, the proposed architecture, the requirements, and the project methodology. The next chapter presents the literature review needed to understand the AI and Data concepts used later in the report.

2 | Literature Review: Data and Artificial Intelligence Foundations

Contents

2.1	Introduction	12
	Chapter Overview	12
2.2	Data Science and Artificial Intelligence	12
2.3	Machine Learning	12
2.4	Deep Learning	13
2.5	Natural Language Processing	13
2.5.1	Rule-Based NLP	13
2.5.2	Transformer-Based NLP	13
2.5.3	Hybrid NLP Strategy	14
2.6	Computer Vision	14
2.7	Object Detection and YOLO	14
2.8	OCR and Text Extraction	15
2.9	Behavior-Driven Development and Gherkin	15
2.10	Browser Automation	15
2.11	Hybrid Intelligent Testing	15
2.12	Conclusion	16

2.1 Introduction

Before presenting the implementation of **VisionTest-AI**, it is important to introduce the main concepts that support the project. The system combines several fields: Data Science, Machine Learning, Deep Learning, Natural Language Processing, Computer Vision, OCR, and software test automation. These fields are connected in the project because the agent must read a scenario, understand it, observe a user interface, execute actions, and produce evidence.

Chapter Overview

This chapter gives a short literature review of the AI and Data concepts used in the project. It explains the difference between Data Science, Machine Learning, Deep Learning, NLP, and Computer Vision. It also presents object detection, OCR, browser automation, and the reason why a hybrid approach is suitable for intelligent functional testing.

2.2 Data Science and Artificial Intelligence

Artificial Intelligence is a broad field that aims to build systems capable of performing tasks that usually require human intelligence. These tasks can include understanding language, recognizing objects, making decisions, or adapting to new situations. In this project, AI is used in a practical way: the system does not try to replace a QA engineer, but to assist the testing process by interpreting scenarios and detecting visual elements.

Data Science is closely related to AI because it focuses on collecting, preparing, analyzing, and using data to support decisions or build models. In **VisionTest-AI**, data appears in different forms: Gherkin text, screenshots, annotations, model outputs, logs, and generated reports. The quality of these data directly affects the reliability of the final system.

2.3 Machine Learning

Machine Learning is a subfield of AI where systems learn patterns from data instead of being programmed only with fixed rules. A machine learning model receives examples during training and then uses what it learned to make predictions on new inputs.

In general, machine learning can be divided into three families:

- **Supervised learning:** The model learns from labeled examples. For example, a screenshot region can be labeled as a button, input field, or link.
- **Unsupervised learning:** The model searches for patterns without explicit labels.
- **Reinforcement learning:** The model learns by interacting with an environment and receiving rewards.

This project mainly uses supervised learning in the vision part. The YOLO model is trained using annotated screenshots where UI elements are labeled with bounding boxes.

2.4 Deep Learning

Deep Learning is a branch of machine learning based on neural networks with several layers. These models are effective when the input data is complex, such as text, images, audio, or video. Instead of manually defining all features, deep learning models learn useful representations from data.

Deep learning is important in this project for two reasons. First, transformer-based models such as BERT and DistilBERT are useful for language understanding [6, 20]. Second, convolutional and detection models such as YOLO are useful for detecting visual elements in screenshots [18, 27].

2.5 Natural Language Processing

Natural Language Processing, or NLP, is the field that studies how computers can process human language. It includes tasks such as text classification, entity extraction, question answering, translation, summarization, and semantic understanding.

In **VisionTest-AI**, NLP is used to understand Gherkin scenarios. Gherkin is readable by humans, but it must be transformed into structured actions before it can be executed by a browser. For example, the sentence:

```
When I enter "demo@example.com" in the email field
```

must be interpreted as an input action, with the value `demo@example.com`, targeting the email field.

2.5.1 Rule-Based NLP

A rule-based NLP system uses predefined patterns. For example, if a sentence contains the word “click”, the system may classify it as a click action. This approach is simple, fast, and predictable. However, it becomes limited when users write steps in different ways.

2.5.2 Transformer-Based NLP

Transformer models improved many NLP tasks because they can capture context in a sentence. BERT introduced a powerful language representation method based on bidirectional context [6]. DistilBERT is a lighter version designed to keep much of BERT’s performance while reducing model size and inference cost [20]. The project uses this family of models because the system must classify user intentions from short natural language steps.

2.5.3 Hybrid NLP Strategy

A fully rule-based system is too rigid, while a fully AI-based system may be difficult to control when confidence is low. For this reason, **VisionTest-AI** uses a hybrid strategy: AI helps classify intent, while deterministic rules and regular expressions are used for reliable extraction of values, URLs, and quoted text.

2.6 Computer Vision

Computer Vision is the field that allows computers to analyze visual information. It includes image classification, object detection, segmentation, OCR, and visual similarity. In this project, computer vision is used to help the agent observe the user interface when selectors are not reliable.

Traditional automation tools mainly interact with the DOM. A vision-based approach adds another source of information: the screenshot. This is useful because a human tester does not need to know the DOM selector of a button; they can recognize the button visually and click it. The objective of the vision module is to bring part of this behavior into the automated system.

2.7 Object Detection and YOLO

Object detection is a computer vision task that identifies both the class and the position of objects in an image. The output is usually a bounding box with a label and a confidence score.

YOLO, meaning “You Only Look Once”, is a family of object detection models designed for fast detection [18]. YOLOv8, provided by Ultralytics, is widely used because it offers a practical API, efficient training, and real-time inference capabilities [27].

In **VisionTest-AI**, YOLO is trained to detect UI elements such as:

- buttons,
- input fields,
- links,
- dropdowns,
- checkboxes.

The detected bounding boxes can then be used to estimate where the agent should click or type when the selector-based approach fails.

2.8 OCR and Text Extraction

OCR, or Optical Character Recognition, transforms text visible in an image into machine-readable text. This is important for UI testing because many elements are identified by their visible label: “Login”, “Submit”, “Search”, or “Cancel”.

Tesseract is one of the most known OCR engines [22, 24]. In the project, OCR is used with screenshot regions in order to enrich visual detections. For example, YOLO may detect a button, while OCR helps determine whether this button contains the text “Log In”.

OpenCV is also useful in the OCR pipeline because it provides image preprocessing operations such as resizing, grayscale conversion, thresholding, and cropping [3]. These operations can improve OCR readability when the image is noisy or low contrast.

2.9 Behavior-Driven Development and Gherkin

Behavior-Driven Development, or BDD, is a development practice where expected behavior is described using examples that can be understood by both technical and non-technical stakeholders. Gherkin is commonly used in BDD because it provides a simple structure based on **Feature**, **Scenario**, **Given**, **When**, and **Then** [4, 28].

This format is useful in the project because it gives the user a readable way to describe functional tests. However, Gherkin alone is not executable. The system must parse it, understand the meaning of each step, map it to an action, and execute it in the browser.

2.10 Browser Automation

Browser automation frameworks allow scripts to simulate user actions. Playwright is used in the project because it supports modern browsers, automatic waiting, screenshots, and reliable end-to-end interactions [13]. It provides the execution engine of the system.

However, Playwright still needs a way to locate elements. This is why **VisionTest-AI** generates several selector candidates and uses visual fallback when these candidates fail. The objective is not to remove Playwright, but to make its execution more adaptive.

2.11 Hybrid Intelligent Testing

The literature and existing tools show that no single technique is sufficient alone. NLP can understand test steps but cannot click a button. Playwright can execute actions but depends on selectors. YOLO can detect UI elements but may confuse visually similar objects. OCR can read labels but may fail on small or low-contrast text.

The project therefore follows a hybrid approach:

- NLP extracts the intention from the Gherkin step.
- Semantic mapping transforms the intention into a browser action.

- Playwright executes the action using selectors.
- YOLO and OCR provide a fallback when selectors fail.
- Reports store the execution evidence.

This combination is more realistic than relying on one technique. It also reflects how a human tester works: reading the scenario, understanding the expected behavior, observing the interface, performing the action, and checking the result.

2.12 Conclusion

This chapter presented the main AI and Data concepts used in **VisionTest-AI**. It introduced Data Science, Machine Learning, Deep Learning, NLP, Computer Vision, object detection, OCR, BDD, Gherkin, and browser automation. These concepts form the scientific and technical foundation of the system. The next chapter focuses on the semantic intelligence module, where Gherkin scenarios are transformed into structured actions and Playwright-compatible mappings.

3 | Semantic Intelligence: NLP and Mapping Module

Contents

3.1	Introduction	19
	Chapter Overview	19
3.2	Objectives of the NLP Module	19
3.3	Gherkin Parsing	20
3.3.1	Parsed Elements	20
3.3.2	Handling And and But	20
3.4	Intent Classification	21
3.4.1	Rule-Based First Version	21
3.4.2	DistilBERT-Based Improvement	21
3.4.3	Confidence Management	21
3.5	Entity Extraction	21
3.5.1	Target and Value Extraction	22
3.5.2	Regex Fallback	22
3.6	Semantic Mapping	22
3.6.1	Selector Candidate Generation	23
3.6.2	Preparing Vision Fallback	23
3.7	API Endpoints of the NLP Module	23
3.7.1	/api/ia/parse-gherkin	23
3.7.2	/api/ia/semantic-mapping	25
3.7.3	/api/ia/generate-test	27
3.8	Generated Playwright Script	28
3.9	Validation and Results	29
3.9.1	Observed Results	29
3.9.2	Error Analysis	30
3.10	Limitations and Possible Improvements	30

3.11 Conclusion	30
------------------------	-----------

3.1 Introduction

After presenting the general requirements and the artificial intelligence foundations of the project, this chapter focuses on the first operational intelligence layer of **VisionTest-AI**: the NLP and semantic mapping module. Its role is to transform a functional scenario written in Gherkin into structured actions that can be executed by Playwright.

The input scenario is readable for a tester, but it is still too abstract for a browser automation engine. A sentence such as `When I click the login button` must be converted into an action type, a target, a possible selector, and optional fallback information. This chapter explains how this transformation is performed in the current prototype.

Chapter Overview

This chapter presents the NLP pipeline used in **VisionTest-AI**. It describes Gherkin parsing, intent classification, entity extraction, semantic mapping, API endpoints, generated Playwright scripts, validation examples, and the main limitations observed during testing.

3.2 Objectives of the NLP Module

The NLP module was designed to solve a simple but important problem: converting business-readable test steps into automation-ready actions. In the context of the project, the module has five main objectives:

- Parse Gherkin scenarios and identify features, scenarios, and steps.
- Detect the intent of each step, such as navigation, click, input, or verification.
- Extract useful entities such as target elements, values, URLs, and expected results.
- Convert the interpreted step into a Playwright-compatible action.
- Provide selector candidates that can be used first by Playwright and later by the vision fallback.

The module is therefore placed at the beginning of the complete pipeline. It receives raw text from the Streamlit interface, sends structured data to the backend services, and prepares the execution plan used by the automation layer.

Figure 3.1 represents the NLP and semantic mapping flow used in the prototype. The diagram was created by the author using Mermaid [12].

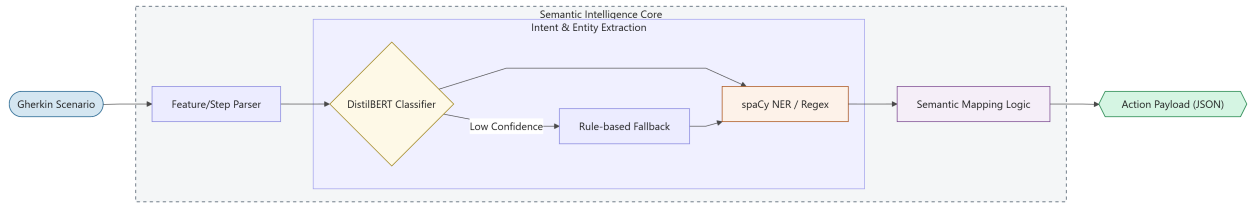


Figure 3.1: NLP and semantic mapping pipeline of VisionTest-AI

3.3 Gherkin Parsing

The parser receives a raw Gherkin scenario and separates it into meaningful blocks. Gherkin was selected because it is widely used in Behavior-Driven Development and is understandable by both technical and non-technical stakeholders [4, 28].

3.3.1 Parsed Elements

The parser extracts three levels of information:

- **Feature:** the general functionality being tested.
- **Scenario:** the specific test case.
- **Steps:** the executable instructions introduced by **Given**, **When**, **Then**, **And**, or **But**.

For example, in a login scenario, the parser identifies the authentication feature, the successful login scenario, and each step related to navigation, typing credentials, clicking a button, and checking the expected result.

Table 3.1 shows how raw Gherkin lines are separated into keywords, step text, and initial interpretation.

Table 3.1: Example of parsed Gherkin steps

Keyword	Step Text	Initial Interpretation
Given	I am on the login page	Navigation or initial context
When	I enter admin in the username field	Input action
And	I click the login button	Click action
Then	I should see the dashboard	Verification action

3.3.2 Handling And and But

The keywords **And** and **But** do not define a new action category by themselves. Their meaning depends on the previous step. For this reason, the parser keeps the original keyword for traceability, but the semantic interpretation is based on the content of the sentence. This avoids treating all **And** steps in the same way.

3.4 Intent Classification

Intent classification identifies what the user wants to do in each step. In the current version, the system supports the most frequent actions required for functional web testing.

Table 3.2 presents the supported intents and their corresponding automation behavior.

Table 3.2: Supported intents in the NLP module

Intent	Playwright Action	Example Step
NAVIGATION	<code>goto</code>	Given I am on <code>https://example.com</code>
ACTION_CLICK	<code>click</code>	When I click the login button
ACTION_TYPE	<code>fill</code>	When I enter admin in the username field
VERIFICATION	<code>expect</code>	Then I should see the dashboard

3.4.1 Rule-Based First Version

The first implementation used deterministic rules. Words such as `open`, `go to`, and `navigate` were mapped to navigation. Words such as `click`, `press`, and `select` were mapped to click actions. This approach was fast and easy to debug, but it was sensitive to wording variation.

3.4.2 DistilBERT-Based Improvement

To improve flexibility, the second version introduced a lightweight Transformer-based classifier using DistilBERT [20]. This model is smaller than BERT [6], which makes it more suitable for a prototype that also includes browser automation and computer vision services.

In practice, the project uses a hybrid strategy. The model is used when the sentence is clear enough, while deterministic rules remain available as a fallback. This keeps the system understandable and avoids relying only on probabilistic output.

3.4.3 Confidence Management

Each interpreted step is associated with a confidence score. When the score is high, the predicted intent is accepted directly. When the score is low or when the step contains explicit patterns such as quoted values or URLs, rule-based extraction is used to protect execution reliability. This design is important because a wrong semantic decision may generate an incorrect browser action.

3.5 Entity Extraction

After detecting the intent, the module extracts the entities required for execution. The most important entities are the target element, the value to type, the URL, and the expected result.

Table 3.3 summarizes the main entities extracted from the interpreted Gherkin steps.

Table 3.3: Main entities extracted from Gherkin steps

Entity	Role	Example
Target	UI element to interact with.	email field, login button
Value	Text to type or value to verify.	demo@example.com
URL	Page to open.	https://www.facebook.com/
Element type	Expected UI component category.	input, button, link

3.5.1 Target and Value Extraction

The target is usually detected from expressions such as `in the email field`, `on the login button`, or `the dashboard`. The value is commonly found inside quotes or between verbs and target expressions. For example:

`When I enter demo@example.com in the email field`

The extracted value is `demo@example.com`, and the extracted target is `email field`. The element type is inferred as `input` because the target contains the word `field`.

3.5.2 Regex Fallback

Regular expressions are used for information that must be exact, especially URLs and quoted values. This fallback is intentionally simple because it protects the system from avoidable mistakes. For example, when the user writes a value between quotes, the system should not paraphrase or modify it.

3.6 Semantic Mapping

Semantic mapping converts the interpreted step into a browser action. This part of the system is not only linguistic; it also prepares technical information for execution.

Table 3.4 shows how each semantic intent is translated into a Playwright-compatible method.

Table 3.4: Mapping between semantic intents and Playwright methods

Intent	Method	Generated Information
NAVIGATION	<code>goto</code>	URL to open
ACTION_CLICK	<code>click</code>	Selector candidates for a button, link, or role
ACTION_TYPE	<code>fill</code>	Selector candidates and input value
VERIFICATION	<code>expect</code>	Expected text or visible element

3.6.1 Selector Candidate Generation

For each target, the mapping service generates several selector candidates. This avoids depending on one fragile selector. For an email field, candidates may include `input[name='email']`, `input#email`, placeholders, partial name matches, partial ID matches, and label-based XPath selectors. During execution, Playwright tries the strongest selector first and can move to alternative candidates when needed.

3.6.2 Preparing Vision Fallback

The semantic output is also useful for the vision module. When a Playwright selector fails, the execution service still knows the target label and the expected element type. This information is later combined with YOLO and OCR detections to search for the element visually.

3.7 API Endpoints of the NLP Module

The semantic intelligence layer is exposed through FastAPI endpoints. This made it easier to test each part independently from the Streamlit interface.

3.7.1 `/api/ia/parse-gherkin`

This endpoint receives a raw Gherkin scenario and returns the parsed structure. It is useful for checking whether the system correctly identifies the feature, scenario, and steps before generating executable actions.

Figure 3.2 shows an example request sent to the parsing endpoint.

POST /api/ia/parse-gherkin Parse Gherkin Endpoint

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "gherkin_text": "Feature: Login\n  Scenario: Successful login\n    When I click the login button"
}
```

Execute

Figure 3.2: Example request of the /api/ia/parse-gherkin endpoint

Figure 3.3 represents the response returned by the endpoint after parsing the Gherkin scenario.

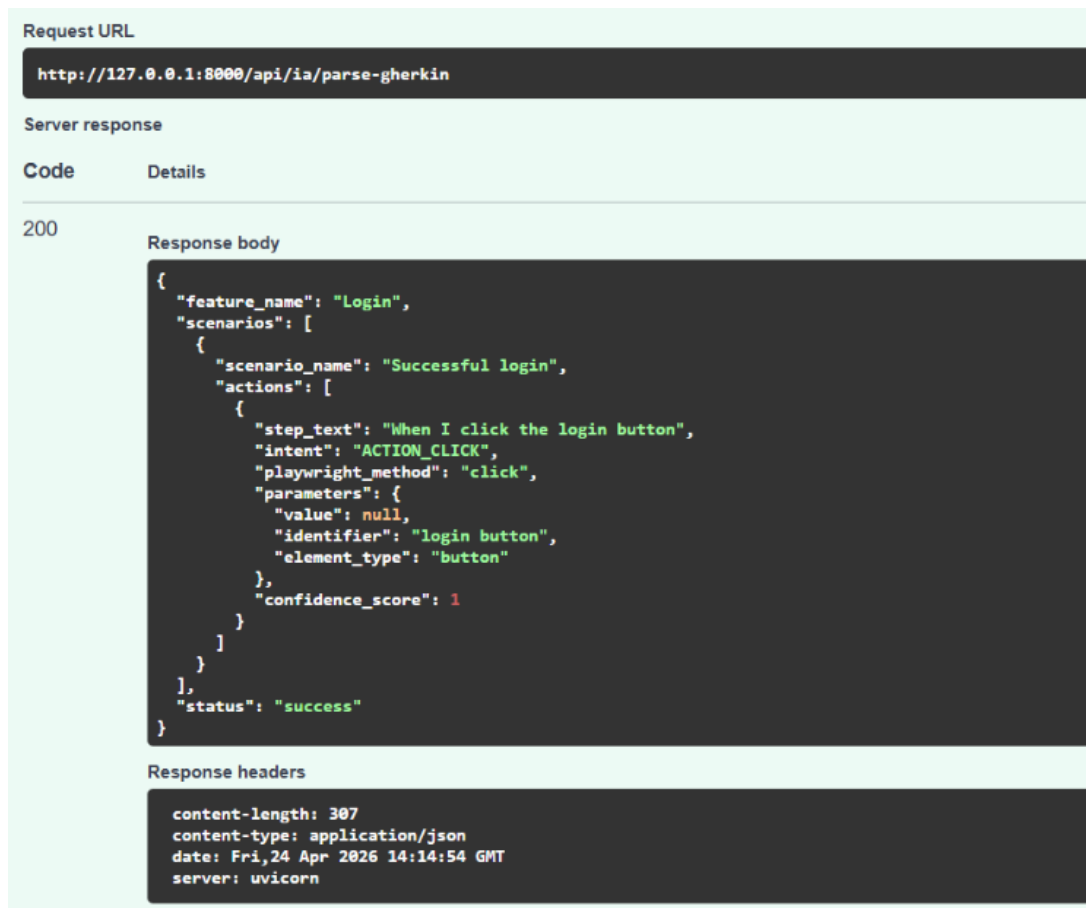


Figure 3.3: Example response of the `/api/ia/parse-gherkin` endpoint

3.7.2 `/api/ia/semantic-mapping`

The `/api/ia/semantic-mapping` endpoint maps one raw step or one NLP payload into an actionable Playwright step. This endpoint was especially useful during debugging because it allowed each semantic decision to be tested separately.

Figure 3.4 shows the request format used to test the semantic mapping endpoint.

POST /api/ia/semantic-mapping Map Semantics

Maps a raw Gherkin step or an NLP payload to an actionable Playwright step.

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | **Schema**

```
{
  "step": "When I enter demo@example.com in the email field",
  "step_text": "When I enter demo@example.com in the email field",
  "intent": "ACTION_TYPE",
  "values": ["demo@example.com"],
  "target": "email field",
  "value": "demo@example.com",
  "element_type": "input"
}
```

Figure 3.4: Example request of the /api/ia/semantic-mapping endpoint

The following response shows an input step where the system extracts the typed value, detects the target as the email field, infers the element type as an input, and generates selector candidates.

Listing 3.1: Semantic mapping response for an input action

```
{
  "step_text": "When I enter demo@example.com in the email field",
  "intent": "ACTION_TYPE",
  "playwright_method": "input",
  "selector": "input[name='email']",
  "value": "demo@example.com",
  "element_type": "input",
  "selector_candidates": [
    "input[name='email']",
    "input#email",
    "input[placeholder*='email']",
    "input[name*='email']",
    "input[id*='email']"
  ]
}
```

```
],  
  "metadata": {  
    "raw_nlp": {  
      "target": "email field",  
      "values": ["demo@example.com"],  
      "element_type": "input"  
    }  
  }  
}
```

3.7.3 /api/ia/generate-test

The `/api/ia/generate-test` endpoint processes a complete Gherkin scenario. Unlike the semantic mapping endpoint, which focuses on one step, this endpoint returns both a generated Playwright script and the list of interpreted actions. Figure 3.5 shows an example

request for generating a Playwright test from a complete scenario.

The screenshot shows a REST client interface for a POST request to `/api/ia/generate-test`. The interface includes a 'Parameters' section with 'Cancel' and 'Reset' buttons, a 'Request body' section with a dropdown set to 'application/json', and a large text area for the request body. The request body is a JSON object with the following content:

```
{  
  "gherkin_text": "Feature: Login to Facebook\n Scenario: Successful login\n  Given I am on  
'https://www.facebook.com/'\n  When I type 'nour.karia@horizon.tn' into the 'email' field\n  And I click the 'Log In' button",  
  "test_name": "facebook_login_test",  
  "base_url": "https://www.facebook.com/"  
}
```

At the bottom of the interface, there are 'Execute' and 'Clear' buttons.

Figure 3.5: Example request of the `/api/ia/generate-test` endpoint

Listing 3.2: Simplified response of the test generation endpoint

```
{
  "status": "success",
  "test_name": "facebook_login_test",
  "scenarios_count": 1,
  "script": "import { test, expect } from '@playwright/test'; ...",
  "actions": [
    {
      "step_text": "Given I am on 'https://www.facebook.com/'",
      "intent": "NAVIGATION",
      "playwright_method": "navigate"
    },
    {
      "step_text": "When I type 'nour.karia@horizon.tn' into the 'email' field",
      "intent": "ACTION_TYPE",
      "playwright_method": "input"
    },
    {
      "step_text": "And I click the 'Log In' button",
      "intent": "ACTION_CLICK",
      "playwright_method": "click"
    }
  ]
}
```

3.8 Generated Playwright Script

The output of semantic mapping can be transformed into a Playwright script [13]. This generated script gives the tester a clear view of what the system understood before execution.

Listing 3.3: Generated Playwright script from the Facebook login scenario

```
import { test, expect } from '@playwright/test';

test("facebook_login_test", async ({ page }) => {
  await page.goto("https://www.facebook.com/");
  await page.locator("input[name='email']").first()
    .fill("nour.karia@horizon.tn");
  await page.getByRole('button', { name: "Log In",
    exact: false }).click();
});
```


3.9 Validation and Results

The validation of this module focused on realistic steps that appear frequently in functional tests: opening a page, filling a field, clicking a button, searching for a product, and checking that a message or dashboard is visible.

Table 3.5 presents examples of correctly interpreted steps during validation.

Table 3.5: Validation examples for the NLP and semantic mapping module

Input Step	Intent	Target	Value
Given I am on <code>https://example.com</code>	Navigation	page URL	<code>https://example.com</code>
When I enter admin in the username field	Input	username field	admin
When I click the login button	Click	login button	–
Then I should see the dashboard	Verification	dashboard	–

3.9.1 Observed Results

The tests showed that the module works well for explicit and well-structured steps. Navigation steps with URLs were extracted reliably. Input steps worked correctly when the target field and the value were both present. Click actions were also mapped correctly when the target was specific, such as `login button` or `search button`.

Table 3.6 summarizes the main mapping results observed during these tests.

Table 3.6: Observed mapping results

Case	Result	Observation
URL navigation	Successful	URLs are easy to detect with regex fallback.
Input with clear field	Successful	Target and value are extracted correctly.
Click with visible button name	Successful	Several selector candidates are generated.
Abstract business step	Partial	The system needs more context to choose an action.
Unsupported interaction	Failed	Upload, drag-and-drop, and hover are not yet mapped.

3.9.2 Error Analysis

Some errors are normal at this prototype stage. They are mainly caused by ambiguity, missing information, or unsupported actions.

Table 3.7 presents representative NLP errors and the correction strategy considered for each case.

Table 3.7: NLP limitations observed during validation

Input Step	Observed Problem	Correction Strategy
I authenticate with valid credentials	Too abstract for direct execution.	Ask for explicit username and password steps.
I choose the first result	Depends on dynamic page state.	Add ranking logic using DOM and visual order.
I upload a file	Unsupported action.	Add an upload intent and file chooser mapping.
I confirm the operation	Ambiguous target.	Use popup context, OCR text, or a clearer target name.

3.10 Limitations and Possible Improvements

The current NLP module provides a solid foundation, but it remains limited to the most common functional testing actions. Natural language can still be ambiguous, especially when the user writes business-level steps without concrete targets. The module is also mainly oriented toward English scenarios; supporting French or mixed-language scenarios would require more examples and additional extraction rules. Future improvements include expanding the intent list, improving multilingual support, adding confidence thresholds in the interface, using previous execution reports to improve selector generation, and asking the user for confirmation when a mapping is uncertain.

3.11 Conclusion

This chapter presented the semantic intelligence layer of **VisionTest-AI**. The NLP module parses Gherkin scenarios, classifies step intents, extracts useful entities, and maps the result into Playwright-compatible actions. The validation examples show that the module works well for explicit functional steps such as navigation, input, click, and verification.

The chapter also showed why a hybrid approach is useful. Transformer-based classification improves flexibility, while deterministic fallback rules protect exact information such as URLs and typed values. The output of this module becomes the input of the execution service and prepares the target information used by the visual fallback. The next chapter presents this visual perception layer, where YOLOv8 and OCR are used to detect UI elements and support self-healing execution.

4 | Visual Perception and Self-Healing Execution

Contents

4.1	Introduction	33
	Chapter Overview	33
4.2	Objective of the Vision Module	33
4.3	Dataset Preparation: From Failed Attempts to DOM-Assisted Labeling	34
4.3.1	The Challenge of Building a UI Dataset	34
4.3.2	First Attempt: Searching for Ready-Made Datasets	34
4.3.3	Second Attempt: Playwright Screenshot Generation	35
4.3.4	Third Attempt: Manual Annotation with Roboflow and LabelImg	35
4.3.5	Limitations of Classic Auto-Labeling	36
4.3.6	Final Solution: DOM-Assisted Auto-Labeling	37
4.3.7	UI Element Classes	38
4.3.8	Dataset Split and Preparation for Training	39
4.4	YOLOv8n Training	39
4.4.1	Why YOLOv8n	39
4.4.2	Training Configuration	40
4.4.3	Loss Function Analysis	40
4.4.4	Training Metrics	41
4.4.5	Model Evaluation	41
4.5	OCR Pipeline	42
4.5.1	Role of OCR in the Vision Module	42
4.5.2	OpenCV Preprocessing	42
4.5.3	Tesseract OCR	43
4.5.4	Text Cleaning and Matching	43
4.6	Vision API Implementation	43

4.6.1	<code>/api/ia/analyze-screenshot</code>	43
4.6.2	Screenshot Input	44
4.6.3	Bounding Box Detection	44
4.6.4	OCR Enrichment	44
4.6.5	Returned JSON Structure	45
4.7	Self-Healing Execution	45
4.7.1	Plan A: Playwright Selector Execution	45
4.7.2	Plan B: YOLO + OCR + Coordinate-Based Action	46
4.7.3	Retry Mechanism	46
4.7.4	Popup and Dialog Handling	46
4.8	Results and Observations	47
4.8.1	Successful Detection Cases	47
4.8.2	Successful Self-Healing Cases	47
4.8.3	Failed Cases	48
4.8.4	Error Analysis	48
4.9	Conclusion	49

4.1 Introduction

After presenting the semantic intelligence layer of **VisionTest-AI**, this chapter focuses on the visual perception layer. While the NLP module is responsible for understanding what the user wants to test, the vision module is responsible for observing the application interface and detecting the elements that can be used during execution.

Traditional web automation tools mainly interact with the Document Object Model (DOM). This approach is powerful when selectors are stable, but it becomes fragile when the interface changes, when elements are dynamically generated, or when the tested page contains components that are difficult to locate using classical selectors. For this reason, **VisionTest-AI** introduces a visual fallback mechanism based on object detection and OCR.

The objective of this chapter is to present the complete construction of the vision module. It describes the dataset preparation process, including the difficulties encountered during data collection and annotation. It then explains the training of the YOLOv8n model, the OCR pipeline, the vision API implementation, and the self-healing execution mechanism. Finally, it presents realistic observations, successful cases, failed cases, and limitations identified during testing.

Chapter Overview

This chapter follows the complete visual workflow. It starts with dataset preparation and the annotation difficulties encountered during the project. It then presents YOLOv8n training, OCR preprocessing, the vision API, the self-healing execution mechanism, and the real observations obtained during testing.

4.2 Objective of the Vision Module

The vision module was designed to make the agent more resilient when classical selector-based automation fails. In a normal execution flow, the system first tries to interact with the page using Playwright selectors generated by the semantic mapping module. However, if these selectors fail, the system needs another way to locate the target element.

The main objective of the vision module is therefore to analyze screenshots and detect visible UI elements such as buttons, input fields, links, dropdowns, and checkboxes. Once these elements are detected, OCR is used to extract visible text and improve the matching between the semantic target and the visual elements.

The module has four main responsibilities:

- Detect UI elements from screenshots using YOLOv8n.
- Extract visible text using OCR.
- Match the semantic target from the NLP module with the detected visual elements.
- Provide coordinates that can be used by Playwright for fallback execution.

This approach does not replace Playwright selectors. Instead, it complements them. The final execution strategy is based on two plans: Plan A uses classical selectors, while Plan B uses visual detection and coordinate-based interaction when selectors fail.

4.3 Dataset Preparation: From Failed Attempts to DOM-Assisted Labeling

The preparation of the vision dataset was one of the most difficult and time-consuming parts of the project. Unlike common object detection tasks, where public datasets are often available, UI element detection requires screenshots containing realistic web components such as buttons, input fields, links, dropdowns, and checkboxes.

At the beginning of the project, the objective seemed simple: collect enough screenshots and annotate the visible interface elements. However, this phase quickly revealed several practical difficulties. Ready-made datasets were not diverse enough, manual annotation was slow, and automatic labeling tools were not immediately reliable for complex web interfaces.

For this reason, the dataset preparation evolved through several attempts before reaching the final solution based on DOM-assisted auto-labeling.

4.3.1 The Challenge of Building a UI Dataset

Building a dataset for UI element detection is different from building a dataset for natural objects. In traditional object detection, classes such as cars, persons, or animals have relatively stable visual characteristics. In web interfaces, the same class can appear in many different forms.

For example, a button can be a filled rectangle, an outlined element, a rounded component, a text-only link styled as a button, or an icon button. Similarly, input fields can have different borders, placeholders, icons, labels, and colors depending on the design system used by the website.

This visual diversity makes the dataset preparation phase critical. If the dataset is too narrow, the trained model may only recognize elements similar to those seen during training and fail on unseen websites.

4.3.2 First Attempt: Searching for Ready-Made Datasets

The first attempt consisted of searching for existing datasets containing annotated UI elements. The goal was to save time by reusing a public dataset instead of building one from scratch.

However, this approach was not sufficient for the project. Most available datasets were either focused on mobile applications, contained synthetic interfaces, or did not include enough variation in real web layouts. Some datasets also used class names that were not aligned with the needs of **VisionTest-AI**.

The main limitation was the lack of diversity. A model trained on a limited dataset may perform well on similar screenshots, but fail when tested on websites with different themes, resolutions, layouts, or component libraries. Since the objective of the project is to support functional testing on different web applications, dataset diversity was essential.

4.3.3 Second Attempt: Playwright Screenshot Generation

The second attempt was to generate screenshots using **Playwright**. This approach made it possible to open web pages automatically, capture screenshots, and create a first collection of realistic interface images.

This method was useful because it allowed the collection of screenshots from real websites and different viewport sizes. It also helped capture desktop and responsive layouts. However, screenshot generation alone did not solve the main problem: the images still needed accurate annotations.

Without reliable labels, the screenshots could not be used directly for YOLO training. This led to the next step: manual and semi-automatic annotation.

4.3.4 Third Attempt: Manual Annotation with Roboflow and LabelImg

Manual annotation was tested using tools such as **Roboflow** [19] and **LabelImg** [26]. These tools allow bounding boxes to be drawn around UI elements and exported in YOLO format.

Although this approach produced accurate labels for some screenshots, it quickly became difficult to scale. Annotating each button, input field, link, and checkbox manually required a lot of time. This was especially challenging for pages containing many repeated links, menus, forms, and small clickable elements.

Another difficulty was annotation consistency. For example, deciding whether a clickable text should be labeled as a link or a button was not always obvious. Small differences in labeling rules can affect the learning process and reduce model performance.

For these reasons, manual annotation was useful for validation and correction, but it was not sufficient as the main dataset creation strategy.

Figure 4.1 shows one of the manual annotation attempts performed during the dataset preparation phase.

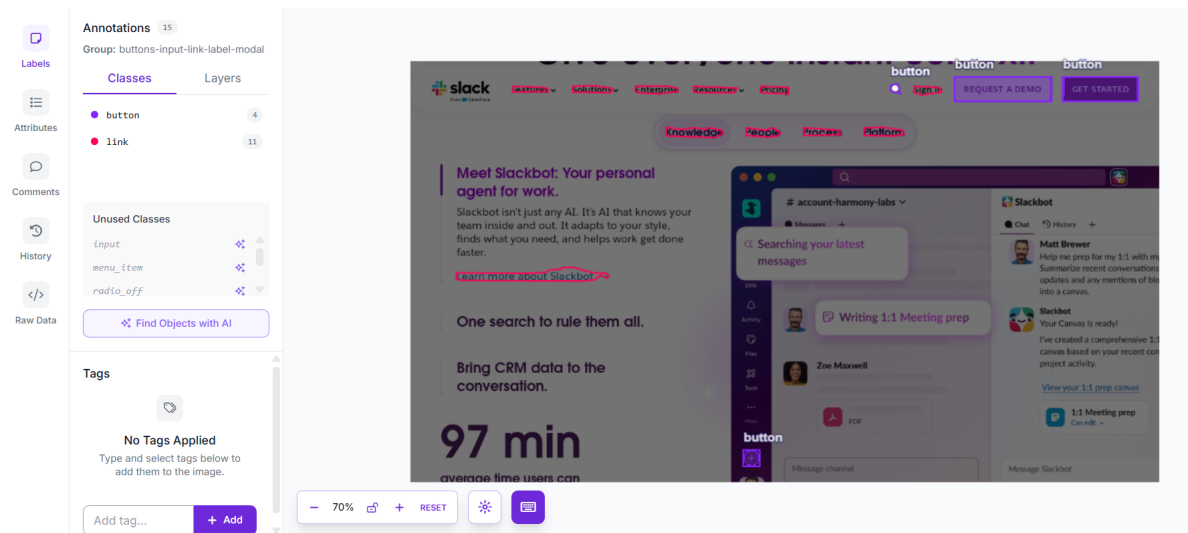


Figure 4.1: Manual annotation attempt using Roboflow

4.3.5 Limitations of Classic Auto-Labeling

Auto-labeling tools were also tested to accelerate the annotation process. However, the results were not immediately reliable. In several cases, the generated boxes were inaccurate, incomplete, or assigned to the wrong class.

This problem is understandable because UI elements are visually diverse. A button may look like a simple text label, an icon, or a traditional rectangle. Input fields may appear with or without borders. Links may be mixed with paragraphs or navigation menus.

The first auto-labeling attempts therefore required significant manual correction. Instead of fully solving the problem, they showed that a more context-aware strategy was necessary.

Figure 4.2 represents an example where automatic labeling did not produce sufficiently reliable annotations.

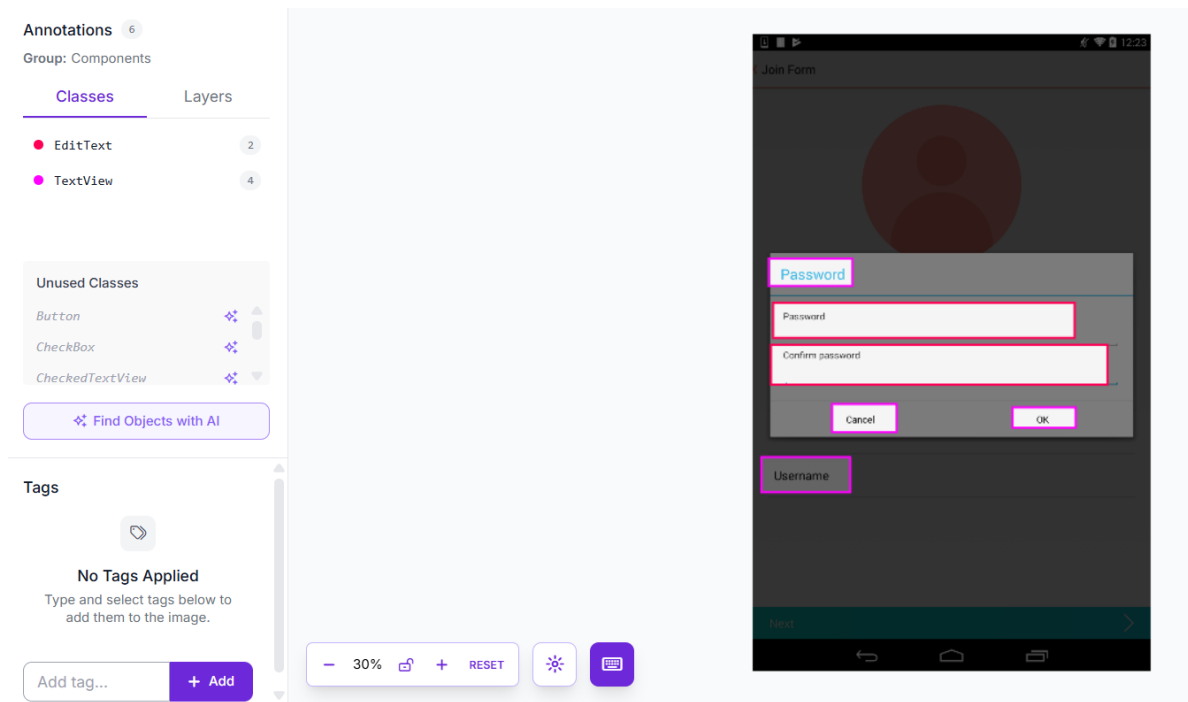


Figure 4.2: Example of incorrect or incomplete auto-labeling result

4.3.6 Final Solution: DOM-Assisted Auto-Labeling

The final solution was to use a DOM-assisted auto-labeling strategy. Instead of relying only on visual auto-labeling, the system used information extracted from the structure of the web page to generate more accurate bounding boxes.

With Playwright, it is possible to inspect DOM elements, retrieve their bounding boxes, and extract useful attributes such as tag name, role, type, placeholder, text content, and accessibility labels. This information was used to identify candidate UI elements and assign them to the appropriate classes.

For example:

- `button` tags and elements with `role="button"` were labeled as buttons.
- `input` and `textarea` elements were labeled as input fields.
- `a` tags were labeled as links.
- `select` elements were labeled as dropdowns.
- `input[type="checkbox"]` elements were labeled as checkboxes.

This approach significantly improved the quality of the annotations. The labels became more consistent because they were generated from the actual structure of the page instead of being guessed only from pixels. Manual correction was still necessary in some cases, but the amount of manual work was reduced.

Figure 4.3 illustrates the DOM-assisted labeling strategy adopted after the first failed attempts. The diagram was created by the author using Mermaid [12].

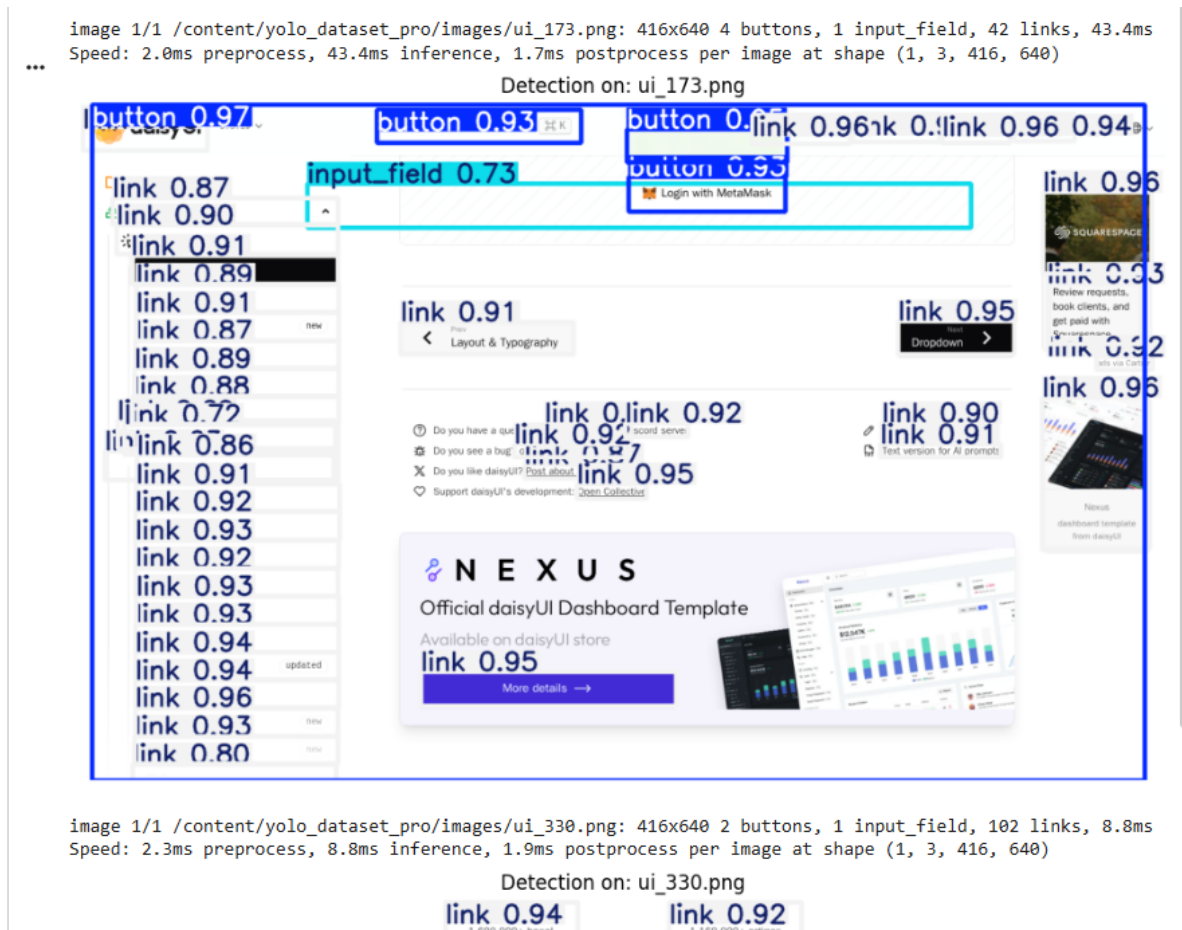


Figure 4.3: DOM-assisted labeling approach used to generate UI annotations

4.3.7 UI Element Classes

The dataset focused on the most common interactive UI elements needed for functional testing. The selected classes were:

- **Button:** Used for actions such as login, submit, search, confirm, or cancel.
- **Input:** Used for text fields, search bars, email fields, password fields, and text areas.
- **Link:** Used for navigation elements and clickable text.
- **Dropdown:** Used for select menus and option lists.
- **Checkbox:** Used for binary selections and form validation.

These classes were selected because they cover the most common user interactions in web functional testing.

Table 4.1 describes the UI classes used to annotate the dataset.

Table 4.1: UI classes used in the vision dataset

Class	Description	Examples
Button	Clickable component that triggers an action.	Login, Submit, Search
Input	Field where the user can enter text.	Email, Password, Search field
Link	Clickable text or navigation element.	Menu link, article link
Dropdown	Component used to select one option from a list.	Country selector, filter menu
Checkbox	Binary selection component.	Remember me, Accept terms

4.3.8 Dataset Split and Preparation for Training

After annotation, the dataset was exported in YOLO format. The images and labels were divided into training, validation, and test subsets. This split was necessary to evaluate the model on unseen data and reduce the risk of overfitting.

The training set was used to update the model weights, the validation set was used to monitor performance during training, and the test set was reserved for final evaluation.

Table 4.2 explains the role of each dataset subset used during training and evaluation.

Table 4.2: Dataset split used for YOLOv8n training

Subset	Role
Training set	Used to train the model and update its weights.
Validation set	Used during training to monitor generalization and tune parameters.
Test set	Used after training to evaluate the model on unseen examples.

4.4 YOLOv8n Training

After obtaining a more reliable annotated dataset, the next step was to train the object detection model. The objective was to detect UI elements from screenshots and return their class, confidence score, and bounding box coordinates.

4.4.1 Why YOLOv8n

The selected model was **YOLOv8n** [27, 18]. This version was chosen because it offers a good compromise between speed and accuracy. Since the vision module is used during test execution, the model must be fast enough to analyze screenshots without slowing down the whole automation pipeline.

YOLOv8n is also lightweight compared to larger YOLO variants. This makes it more suitable for a prototype that may run in local environments or limited cloud environments.

Table 4.3 compares the YOLOv8 variants considered during model selection.

Table 4.3: YOLOv8 variants and project positioning

Model	Advantage	Reason for Selection
YOLOv8n	Lightweight and fast.	Selected for real-time screenshot analysis.
YOLOv8s	More accurate but heavier.	Considered, but less suitable for lightweight execution.
YOLOv8m/l	Higher capacity models.	Not selected due to resource constraints.

4.4.2 Training Configuration

The training configuration was selected to balance detection quality and computational cost. The model was trained using a standard image size of 640×640 , which is commonly used with YOLO models and preserves enough detail for small UI elements.

Table 4.4 presents the final training configuration used for YOLOv8n.

Table 4.4: YOLOv8n training configuration

Parameter	Value	Justification
Model	YOLOv8n	Lightweight model suitable for real-time detection.
Image size	640	Preserves UI details while keeping inference efficient.
Epochs	100	Allows the model to learn UI element patterns progressively.
Batch size	16	Balanced choice according to available memory.
Task	Object detection	Required to locate UI elements with bounding boxes.

4.4.3 Loss Function Analysis

During training, several losses were monitored:

- **Box loss:** Measures the quality of bounding box localization.
- **Class loss:** Measures the quality of class prediction.
- **DFL loss:** Helps refine bounding box boundaries.

For this project, box localization is particularly important. If a button is detected but the bounding box is inaccurate, the fallback coordinate click may be placed outside the real element. This would reduce the reliability of the self-healing mechanism.

4.4.4 Training Metrics

After training, the model performance was analyzed using the metrics generated by YOLOv8. These metrics include training losses, validation losses, precision, recall, and mean Average Precision.

Figure 4.4 represents the training metrics obtained after the YOLOv8n training process.

```
100 epochs completed in 0.076 hours.
Optimizer stripped from /content/runs/detect/vision_test_ai_final-2/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/vision_test_ai_final-2/weights/best.pt, 6.2MB

Validating /content/runs/detect/vision_test_ai_final-2/weights/best.pt...
Ultralytics 8.4.41 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	64	1785	0.984	0.963	0.992	0.895
button	64	385	0.975	0.971	0.992	0.895
link	64	1400	0.993	0.955	0.992	0.894

```
Speed: 0.1ms preprocess, 2.7ms inference, 0.0ms loss, 6.9ms postprocess per image
Results saved to /content/runs/detect/vision_test_ai_final-2
```

Figure 4.4: YOLOv8n training metrics: loss curves, precision, recall, and mAP evolution

The curves provide insight into the behavior of the model during training. A decreasing loss indicates that the model is learning useful visual patterns, while stable validation metrics suggest that the model is not only memorizing the training examples.

4.4.5 Model Evaluation

The evaluation focused on whether the model could correctly detect the most important UI components for functional testing. Buttons and input fields were especially important because they are directly used during click and input actions.

The results showed that the model was able to detect several common components correctly. However, some classes remained more difficult, especially when they appeared in small sizes, low contrast, or visually similar styles.

Figure 4.5 shows the confusion matrix used to analyze the model behavior across UI classes.

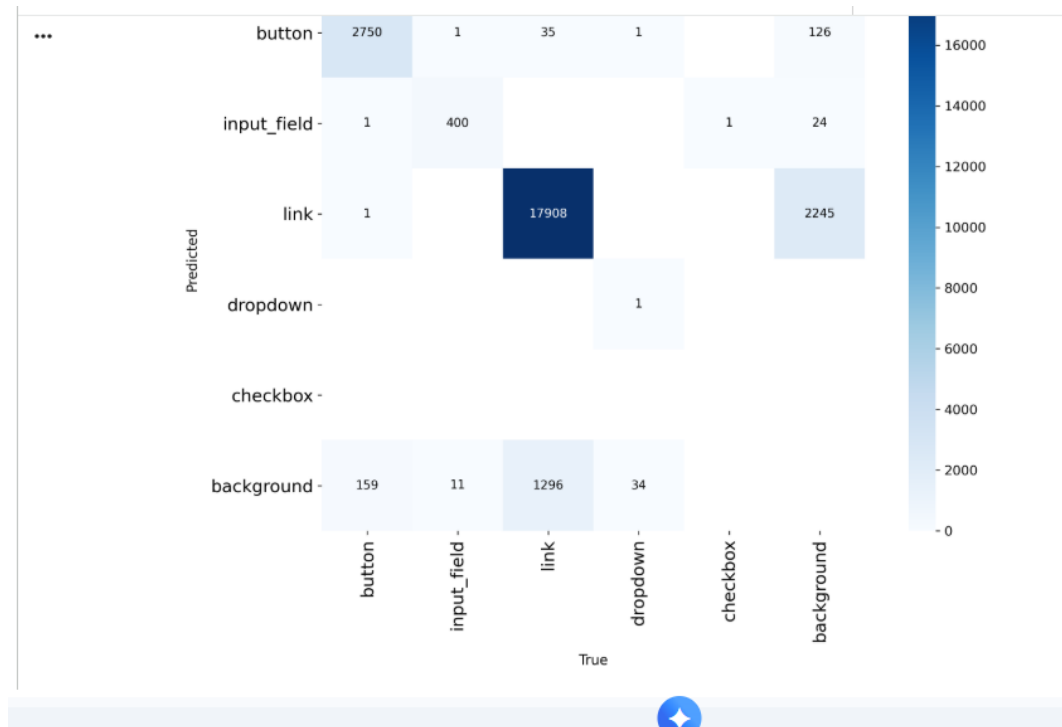


Figure 4.5: Confusion matrix of the trained YOLOv8n model

4.5 OCR Pipeline

Object detection alone is not enough for self-healing execution. YOLO can detect that an element is a button or an input field, but it does not always know which button is the correct one. For example, a page can contain several buttons such as *Login*, *Cancel*, *Search*, and *Submit*.

For this reason, OCR was integrated into the vision pipeline. OCR helps extract visible text from the screenshot and associate it with detected elements.

4.5.1 Role of OCR in the Vision Module

The role of OCR is to enrich visual detections with textual information. When the NLP module asks for the `login` button, the vision module can use OCR to find detected buttons whose visible text is close to `login`.

This improves the matching process between the semantic target and the visual candidates.

4.5.2 OpenCV Preprocessing

Before applying OCR, screenshots can be preprocessed using **OpenCV** [3]. Preprocessing improves the readability of text and may include grayscale conversion, thresholding, noise reduction, or cropping detected regions.

The objective is not to modify the interface, but to make the text easier for OCR to read.

4.5.3 Tesseract OCR

Tesseract OCR [22, 24] was used to extract text from screenshots and detected regions. The extracted text is then cleaned and normalized before being compared with the target extracted from the Gherkin step.

OCR is especially useful for buttons, links, and labels. However, it can be less reliable when the text is small, blurred, partially hidden, or rendered with low contrast.

4.5.4 Text Cleaning and Matching

After OCR extraction, the text is cleaned by removing unnecessary spaces, normalizing case, and filtering irrelevant characters. The cleaned text is then compared with the semantic target using simple matching techniques.

For example, if the target is `login button`, the matching process prioritizes detections containing the text `login`. This helps the vision module choose the most relevant element among several candidates.

4.6 Vision API Implementation

The vision module is exposed through API endpoints that allow screenshots to be analyzed and detections to be returned in JSON format.

4.6.1 /api/ia/analyze-screenshot

The `/api/ia/analyze-screenshot` endpoint receives a screenshot and applies the vision pipeline. It returns detected UI elements, bounding boxes, confidence scores, and OCR information when available.

This endpoint is useful for testing the vision module independently from the full execution pipeline.

Figure 4.6 shows an example request sent to the screenshot analysis endpoint.

Analyze Screenshot API - Example Request

```
POST /api/ia/analyze-screenshot
Content-Type: application/json
```

```
{
  "image_base64":
    "iVBORw0KGgoAAAANSUheUgAABQAAAAALQCAIAAABAH0oBAAAQAE1EQVR4nOz9B4BdV3nuD6+1... <base64
    omitted for readability>",
  "filename": "step_20260504_103710_I_click_the_login_button.png",
  "include_ocr": true,
  "target_text": "login",
  "min_confidence": 0.2
}
```

Figure 4.6: Example request of the `/api/ia/analyze-screenshot` endpoint

4.6.2 Screenshot Input

The screenshot can come from different sources. It can be uploaded by the user, captured during execution, or generated by Playwright when a selector fails.

In the self-healing process, screenshots are usually captured automatically by the execution service at the moment of failure.

4.6.3 Bounding Box Detection

After receiving the screenshot, the YOLOv8n model detects UI elements and returns bounding boxes. Each bounding box contains the position and size of the detected element.

A typical detection includes:

- Class name.
- Confidence score.
- Bounding box coordinates.
- Optional OCR text.

Listing 4.1: Simplified vision detection response

```
{
  "status": "success",
  "detections": [
    {
      "class_name": "button",
      "confidence": 0.86,
      "bbox": [420, 310, 520, 350],
      "text": "Log In"
    },
    {
      "class_name": "input",
      "confidence": 0.91,
      "bbox": [250, 180, 540, 220],
      "text": "Email"
    }
  ]
}
```

4.6.4 OCR Enrichment

After detecting bounding boxes, OCR is applied to selected regions in order to extract visible text. This allows the system to associate semantic labels with visual elements.

For example, a detected button with OCR text `Log In` becomes a stronger candidate when the target is `login button`.

4.6.5 Returned JSON Structure

The returned JSON structure is designed to be consumed by the execution service. It provides enough information to select a candidate and perform a coordinate-based action if needed.

The center of the bounding box can be used as the click coordinate:

$$x_{center} = \frac{x_{min} + x_{max}}{2}$$

$$y_{center} = \frac{y_{min} + y_{max}}{2}$$

This coordinate is then passed to Playwright for fallback execution.

4.7 Self-Healing Execution

The self-healing mechanism is one of the most important parts of **VisionTest-AI**. Its purpose is to continue execution when the primary selector-based strategy fails.

4.7.1 Plan A: Playwright Selector Execution

Plan A is the default execution strategy. The system uses selector candidates generated by the semantic mapping module and tries to execute the action with Playwright.

For example, for the step:

`When I click the login button`

the system may try selectors such as:

- `button[name='login']`
- `button[type='submit']`
- `[role='button']:has-text('Log in')`
- `text='login'`

If one selector succeeds, the step is considered passed.

4.7.2 Plan B: YOLO + OCR + Coordinate-Based Action

If Plan A fails, the system activates Plan B. In this case, the execution service captures a screenshot of the current page and sends it to the vision module.

The vision module detects UI elements, extracts OCR text, ranks candidates, and returns the most relevant element. The execution service then uses the center of the selected bounding box to perform a coordinate-based action.

This process allows the system to recover from selector failures in some cases.

Figure 4.7 represents a screenshot captured during a fallback execution, when selector-based execution was not sufficient.

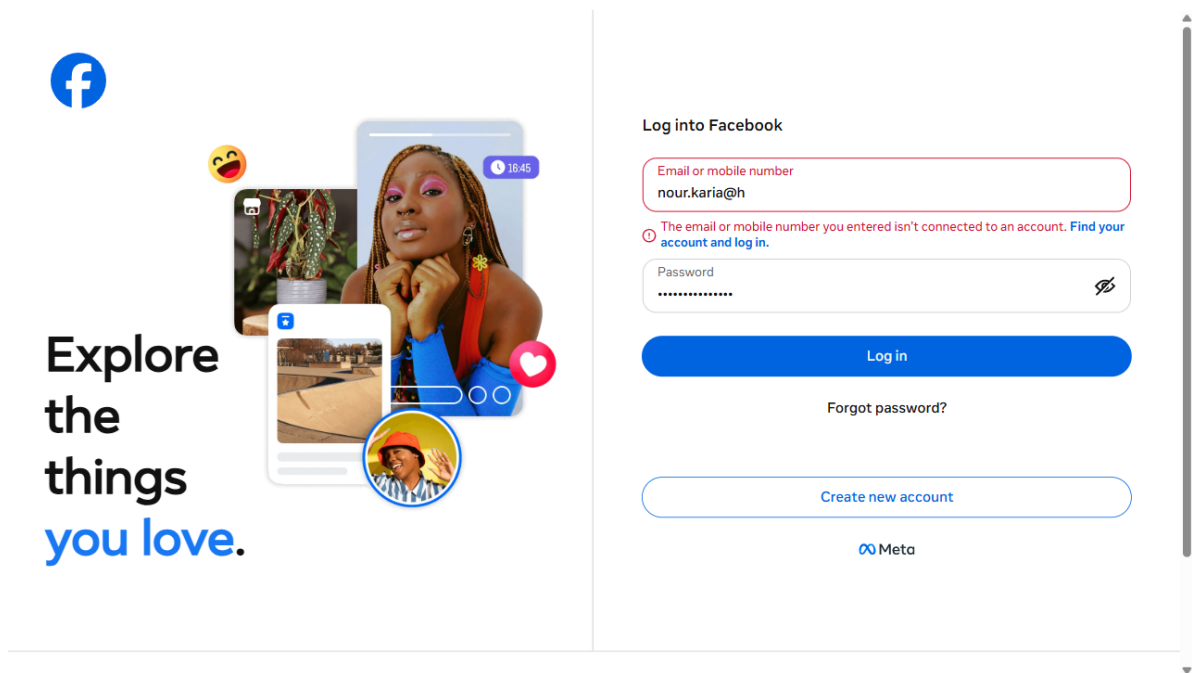


Figure 4.7: Screenshot captured during a fallback execution

4.7.3 Retry Mechanism

Web interfaces are often dynamic. Elements may appear after a delay, popups may block interactions, and content may load asynchronously. For this reason, the execution service includes a retry mechanism.

If an action fails, the system can wait briefly, capture a new screenshot, and try again. This improves robustness in pages where the state changes after loading.

4.7.4 Popup and Dialog Handling

During testing, popups and dialogs were frequently encountered, especially cookie consent dialogs or login overlays. These elements can block the target UI element and cause execution failure.

To handle this, the system attempts to detect common popup buttons such as **Accept**, **Allow**, **Close**, or **OK**. When such elements are detected, the system can interact with them before continuing the original test.

4.8 Results and Observations

The vision module was evaluated through both isolated screenshot analysis and full execution scenarios. The objective was not to claim perfect detection, but to verify whether the module could support the self-healing mechanism in realistic cases.

4.8.1 Successful Detection Cases

The model produced good results on common UI elements such as buttons and input fields. Detection was especially reliable when the elements had clear borders, sufficient contrast, and standard layouts.

Figure 4.8 shows an example of a successful visual detection result.

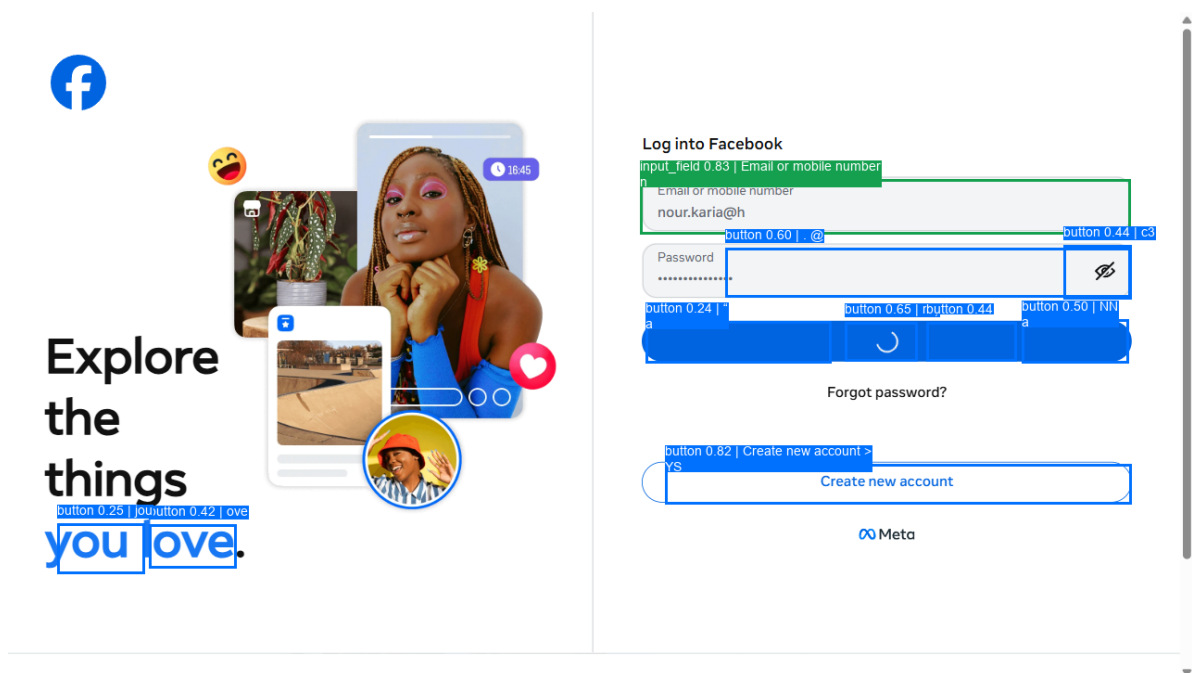


Figure 4.8: Example of successful UI element detection

4.8.2 Successful Self-Healing Cases

The self-healing mechanism was useful when generated selectors failed but the target element was still visible on the screen. In these cases, the system used screenshot analysis and coordinate-based execution to continue the test.

For example, if the selector for a login button failed because the HTML attributes changed, the vision module could still detect the visible button and provide its coordinates.

Table 4.5 summarizes examples where the visual fallback helped continue execution after a selector failure.

Table 4.5: Examples of successful self-healing situations

Situation	Plan A Result	Plan B Result
Login button selector not found	Failed selector lookup	Button detected visually and clicked by coordinates
Search input selector changed	Fill action failed	Input field detected and filled using fallback strategy
Cookie popup blocked page	Main action blocked	Popup button detected and handled before retry

4.8.3 Failed Cases

Some failures were still observed. The most common causes were OCR errors, low contrast, overlapping elements, and ambiguous targets.

For example, if several buttons had similar text or no visible text, the system could choose the wrong candidate. Similarly, if OCR failed to read the label of a small button, the ranking process became less reliable.

Table 4.6 presents the main failed cases observed during the validation of the vision module.

Table 4.6: Observed failed cases in the vision module

Failure Case	Cause	Possible Improvement
Small icon button not detected	Element too small or visually similar to background	Add more icon examples and increase dataset diversity
Wrong button selected	Several candidates had similar visual/textual features	Improve ranking with DOM context and OCR confidence
Text not extracted correctly	OCR failed due to low contrast or small font	Improve preprocessing and OCR configuration
Dropdown not detected reliably	Limited number of dropdown examples in dataset	Add more annotated dropdown samples

4.8.4 Error Analysis

The error analysis shows that the vision module is useful as a fallback mechanism, but it should not be considered a complete replacement for selector-based automation. Playwright selectors remain more precise when the DOM is accessible and stable. The vision module becomes valuable when selectors fail or when the visual state of the page provides stronger information than the HTML structure.

The main improvement needed is dataset diversity. More examples of different UI styles, themes, screen sizes, and component libraries would improve the model’s ability to generalize.

4.9 Conclusion

This chapter presented the visual perception and self-healing execution layer of **VisionTest-AI**. It first described the dataset preparation process, which evolved from searching for ready-made datasets and manual annotation attempts to a more reliable DOM-assisted auto-labeling approach.

The chapter then presented the YOLOv8n training phase, including the choice of model, training configuration, loss analysis, and evaluation metrics. OCR was also integrated to enrich visual detections with text information and improve target matching.

Finally, the chapter explained the self-healing execution mechanism based on Plan A and Plan B. Plan A relies on Playwright selectors, while Plan B uses YOLO, OCR, and coordinate-based actions when selectors fail.

The results showed that the vision module can improve robustness in some realistic situations, especially when the target element remains visible but selector-based execution fails. However, the module still has limitations related to dataset diversity, OCR reliability, and ambiguous visual targets. The next chapter will focus on the complete system integration, reporting, deployment attempts, and final results of the project.

5 | Integration, Reporting, Deployment and Final Results

Contents

5.1	Introduction	52
	Chapter Overview	52
5.2	Complete Pipeline Integration	52
5.2.1	User Submits a Gherkin Scenario	52
5.2.2	NLP Parsing and Semantic Mapping	53
5.2.3	Playwright Execution	53
5.2.4	Vision Fallback When Selectors Fail	53
5.2.5	Report Generation	53
5.3	Backend Implementation	54
5.3.1	FastAPI Routers	54
5.3.2	Services Layer	55
5.3.3	Schemas and Validation	55
5.3.4	Error Handling and Logs	55
5.4	Reporting System	55
5.4.1	JSON Report Generation	56
5.4.2	HTML Report Generation	56
5.4.3	Screenshot Evidence	57
5.4.4	Report API Endpoints	57
5.4.5	Streamlit Report Viewer	58
5.5	User Interface	58
5.5.1	Scenario Studio	58
5.5.2	Script Generation View	59
5.5.3	Screenshot Analyzer	60
5.5.4	Reports Page	60
5.6	Deployment Attempts	61

5.6.1	Local Deployment	61
5.6.2	Docker Deployment	61
5.6.3	First Streamlit Cloud Attempt	62
5.6.4	Resource Limit Issue	62
5.6.5	Final Proposed Architecture: Railway Backend and Streamlit UI .	62
5.6.6	CI/CD Integration Strategy	63
5.7	Testing and Validation	64
5.7.1	Unit Tests	64
5.7.2	API Tests	64
5.7.3	End-to-End Test Scenarios	64
5.7.4	Unseen-Site Benchmark	64
5.8	Actual Results	65
5.8.1	Generated JSON Reports	65
5.8.2	Generated HTML Reports	66
5.8.3	Successful Execution Examples	66
5.8.4	Failed Execution Examples	66
5.8.5	Plan A vs Plan B Usage	67
5.8.6	Performance Observations	67
5.9	Limitations	68
5.9.1	Model Accuracy Limitations	68
5.9.2	OCR Limitations	68
5.9.3	Deployment Resource Limitations	68
5.9.4	Lack of PostgreSQL and Redis Integration	68
5.10	Perspectives	68
5.10.1	Improve NLP Multilingual Support	68
5.10.2	Expand the Annotated Dataset	69
5.10.3	Add PostgreSQL and Redis	69
5.10.4	Add Authentication and Project History	69
5.10.5	Improve Cloud Deployment	69
5.11	Conclusion	69

5.1 Introduction

After presenting the semantic intelligence module and the visual perception layer, this chapter focuses on the complete integration of **VisionTest-AI**. The objective is to show how the different modules work together to form a functional testing pipeline, from the submission of a Gherkin scenario to the generation of execution reports.

This chapter presents the full system workflow, the backend implementation, the reporting system, the user interface, deployment attempts, testing strategy, and final results. It also discusses the limitations observed during the project and the possible improvements for future versions.

Unlike the previous chapters, which focused on specific modules, this chapter provides a global view of the final prototype. It shows how NLP parsing, semantic mapping, Playwright execution, vision fallback, and report generation are connected inside the same system.

Chapter Overview

This chapter presents the final prototype from an engineering point of view. It describes the complete pipeline, backend routers and services, reporting system, Streamlit interface, deployment attempts, validation strategy, actual results, limitations, and perspectives.

5.2 Complete Pipeline Integration

The complete pipeline of **VisionTest-AI** starts with a Gherkin scenario written by the user and ends with structured execution reports. The system was designed as a modular pipeline where each component has a specific responsibility.

5.2.1 User Submits a Gherkin Scenario

The first step begins when the user submits a Gherkin scenario through the interface. The scenario describes the functional behavior to test using readable steps such as **Given**, **When**, and **Then**.

An example scenario is:

Feature: Facebook login

Scenario: Invalid login attempt

Given I am on 'https://www.facebook.com/'

When I type 'nour.karia@horizon.tn' into the 'email' field

And I type 'tourbillon1234' into the 'password' field

And I click the 'Log In' button

Then I should see the error message

This scenario is understandable by a human tester, but it must be transformed into structured actions before execution.

5.2.2 NLP Parsing and Semantic Mapping

After receiving the scenario, the backend sends it to the NLP module. This module extracts the feature name, scenario name, and steps. Each step is then analyzed to identify the intent, target, value, and element type.

The semantic mapping module converts these interpreted steps into executable actions. For example, an input step is mapped to a Playwright `fill` action, while a click step is mapped to a Playwright `click` action.

5.2.3 Playwright Execution

The execution service uses Playwright to interact with the browser. It opens the target page, executes each action, captures screenshots, and records the status of every step.

The primary execution strategy is selector-based. This means that the system first tries to interact with elements using generated selector candidates such as role selectors, text selectors, CSS selectors, or XPath expressions.

5.2.4 Vision Fallback When Selectors Fail

If all selector candidates fail, the system activates the visual fallback strategy. In this case, the execution service captures a screenshot of the current page and sends it to the vision module.

The vision module analyzes the screenshot using YOLOv8n and OCR. It detects UI elements, extracts visible text, ranks possible candidates, and returns coordinates that can be used for a fallback action.

This mechanism is called self-healing execution because it allows the system to recover from some selector failures.

5.2.5 Report Generation

At the end of execution, the reporting service generates structured reports. These reports contain the execution status, the list of executed steps, screenshots, errors, execution duration, and the strategy used for each step.

Two report formats are generated:

- **JSON report:** Used for structured storage and future processing.
- **HTML report:** Used for human-readable visualization.

Figure 5.1 represents the complete execution pipeline of **VisionTest-AI**, from scenario submission to report generation. The diagram was created by the author using Mermaid [12].

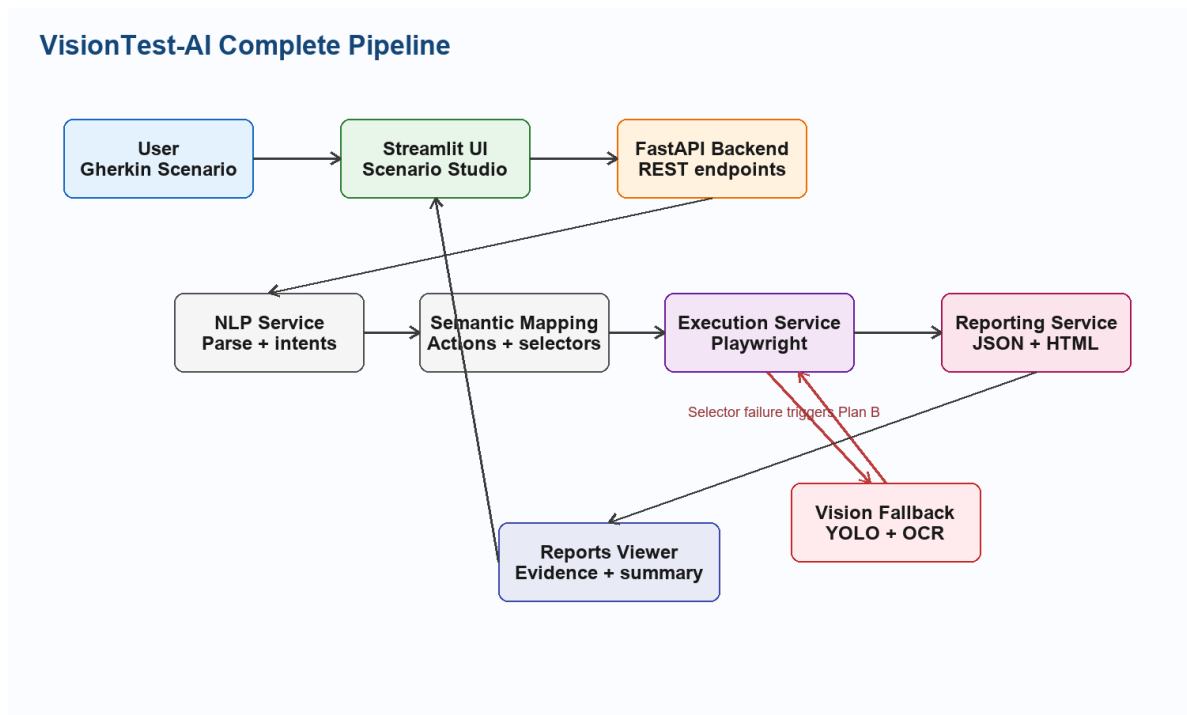


Figure 5.1: Complete execution pipeline of VisionTest-AI

5.3 Backend Implementation

The backend was implemented using **FastAPI** [8]. It exposes the different services of the system through REST API endpoints and coordinates the communication between the NLP, mapping, execution, vision, and reporting modules.

5.3.1 FastAPI Routers

The backend is organized using routers. Each router is responsible for a specific part of the system.

Table 5.1 summarizes the main FastAPI routers implemented in the backend.

Table 5.1: Main FastAPI routers of VisionTest-AI

Router	Responsibility
<code>nlp_parsing.py</code>	Parses Gherkin scenarios and returns structured actions.
<code>semantic_mapping.py</code>	Maps interpreted steps to executable Playwright actions.
<code>test_generation.py</code>	Generates Playwright scripts from Gherkin scenarios.
<code>test_execution.py</code>	Executes test scenarios using Playwright.
<code>ql_detection.py</code>	Analyzes screenshots and detects UI elements.
<code>reporting.py</code>	Lists, retrieves, and displays generated reports.

This organization makes the backend easier to maintain and extend.

5.3.2 Services Layer

The business logic is separated into services. This avoids placing complex logic directly inside API endpoints.

The main services are:

- **NLP service:** Handles Gherkin parsing and step interpretation.
- **Mapping service:** Generates Playwright actions and selector candidates.
- **Execution service:** Runs the browser automation pipeline.
- **Vision service:** Detects UI elements from screenshots.
- **Report service:** Saves JSON and HTML reports.
- **Generator service:** Produces Playwright scripts.

This separation improves modularity and makes each component easier to test independently.

5.3.3 Schemas and Validation

The backend uses **Pydantic** [15] schemas to validate input and output data. This ensures that API requests follow the expected structure and that responses remain consistent.

For example, Gherkin parsing requests contain the raw scenario text, while execution reports contain structured information such as status, duration, steps, screenshots, and metadata.

Using schemas also makes the API documentation clearer through the automatic Swagger interface generated by FastAPI.

5.3.4 Error Handling and Logs

Error handling was an important part of the implementation. During test execution, several types of errors can occur: selector failure, timeout, browser issue, OCR failure, or unexpected page behavior.

The system records these errors and includes them in the final report instead of hiding them. This is important because a test report should not only say that a scenario failed, but also explain where and why it failed.

Logs were also used during development to understand pipeline behavior and debug failures.

5.4 Reporting System

The reporting system is one of the most important parts of the final prototype. It provides evidence of execution and allows the user to analyze the result of each test.

5.4.1 JSON Report Generation

The JSON report stores the execution result in a structured format. It contains information such as the execution ID, feature name, scenario name, status, start time, end time, duration, executed steps, screenshots, and summary metrics.

A simplified example of a JSON report structure is shown below:

Listing 5.1: Simplified JSON report structure

```
{
  "execution_id": "b25635cb-45e4-4c96-8208-ceca0ebf4fc2",
  "feature_name": "Facebook login",
  "scenario_name": "Invalid login attempt",
  "status": "passed",
  "summary": {
    "total_steps": 5,
    "passed_steps": 5,
    "failed_steps": 0,
    "plan_a_steps": 4,
    "plan_b_steps": 1
  },
  "steps": [
    {
      "step_text": "When I click the login button",
      "action": "click",
      "status": "passed",
      "plan_used": "plan_a_selector_playwright",
      "screenshot_path": "reports/screenshots/step_login"
    }
  ]
}
```

The JSON format is useful because it can later be stored in a database, analyzed automatically, or used to generate dashboards.

5.4.2 HTML Report Generation

The HTML report provides a readable view of the execution. It is designed for users who want to quickly understand the result without reading raw JSON.

The report displays:

- General execution status.
- Feature and scenario names.
- Execution duration.

- Number of passed and failed steps.
- Step-by-step details.
- Screenshots captured during execution.
- Errors and metadata when available.

Figure 5.2 shows an example of an HTML report generated by the reporting module.

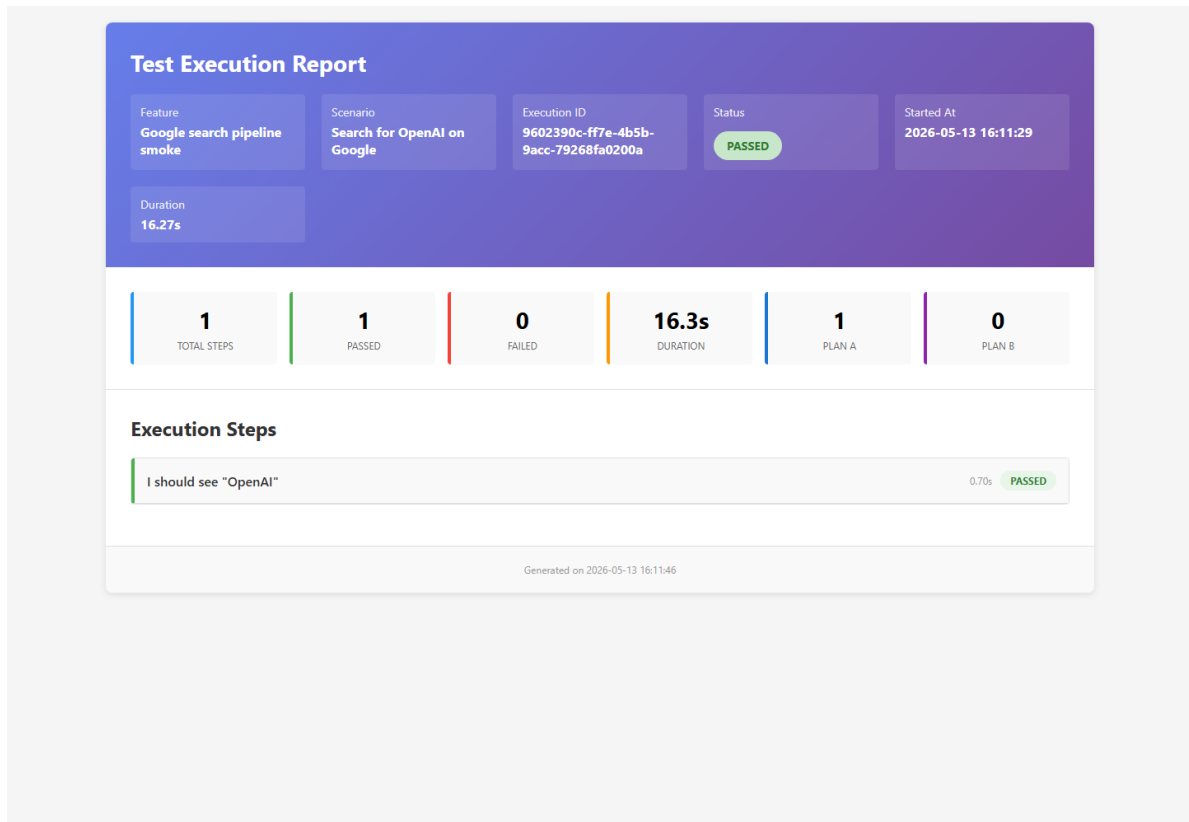


Figure 5.2: Example of an HTML execution report generated by VisionTest-AI

5.4.3 Screenshot Evidence

Screenshots are captured during execution and linked to the report. They provide visual evidence of what happened during each step. This is especially useful when a test fails, because it allows the user to inspect the actual state of the page.

For example, if a login test fails because an error message is displayed, the screenshot allows the tester to confirm the result visually.

5.4.4 Report API Endpoints

The reporting module exposes several API endpoints.

Table 5.2 presents the API endpoints used to list and retrieve generated reports.

Table 5.2: Reporting API endpoints

Endpoint	Description
GET /api/ia/reports	Lists generated reports.
GET /api/ia/reports/{execution_id}	Retrieves a specific JSON report.
GET /api/ia/reports/{execution_id}?format=html	Retrieves a specific HTML report.
GET /api/ia/reports/{execution_id}/summary	Retrieves a compact summary of a report.

5.4.5 Streamlit Report Viewer

The Streamlit interface includes a report viewer. This page allows the user to select a generated report, view the HTML version, inspect the JSON file, and check the execution summary.

This feature makes the reporting system more accessible because the user does not need to manually open files from the project directory.

5.5 User Interface

The frontend of the project was implemented using **Streamlit** [23]. It provides a simple interface for interacting with the backend services.

5.5.1 Scenario Studio

The Scenario Studio allows the user to write or paste a Gherkin scenario. The scenario can then be parsed, transformed into actions, generated as a Playwright script, or executed directly.

This page represents the main entry point of the system.

Figure 5.3 shows the Scenario Studio page used to submit and process Gherkin scenarios.

The screenshot shows the 'Data-AI Test Studio' interface. On the left is a sidebar with a navigation menu containing four items: 'Status', 'Scenario Studio' (which is selected with a blue dot), 'Reports', and 'Screenshot Analyzer'. The main area is titled 'Data-AI Test Studio' with a subtitle 'Streamlit interface for the self-healing functional test automation pipeline.' Below this is the 'Scenario Studio' section. It contains four input fields: 'Feature name' with the value 'Google search pipeline smoke', 'Scenario name' with 'Search for OpenAI on Google', 'Website URL' with 'https://www.google.com/search?q=OpenAI', and a 'Gherkin scenario' text area containing a Gherkin script. At the bottom of the main area are three buttons: 'Parse', 'Generate Script', and 'Run Pipeline' (which is highlighted in blue). In the top right corner, there is a 'Deploy' button with a dropdown arrow.

<< Deploy ▾

Data-AI Test Studio

Streamlit interface for the self-healing functional test automation pipeline.

Scenario Studio

Feature name
Google search pipeline smoke

Scenario name
Search for OpenAI on Google

Website URL
https://www.google.com/search?q=OpenAI

Gherkin scenario

```
Feature: Google search pipeline smoke  
  
Scenario: Search for OpenAI on Google  
Given I navigate to "https://www.google.com/search?q=OpenAI"  
Then I should see "OpenAI"
```

Parse Generate Script Run Pipeline

Figure 5.3: Scenario Studio interface for submitting Gherkin scenarios

5.5.2 Script Generation View

The script generation view displays the Playwright code generated from the submitted Gherkin scenario. This is useful for users who want to inspect, reuse, or manually adapt the generated script.

Figure 5.4 represents the generated Playwright script as displayed in the Streamlit interface.

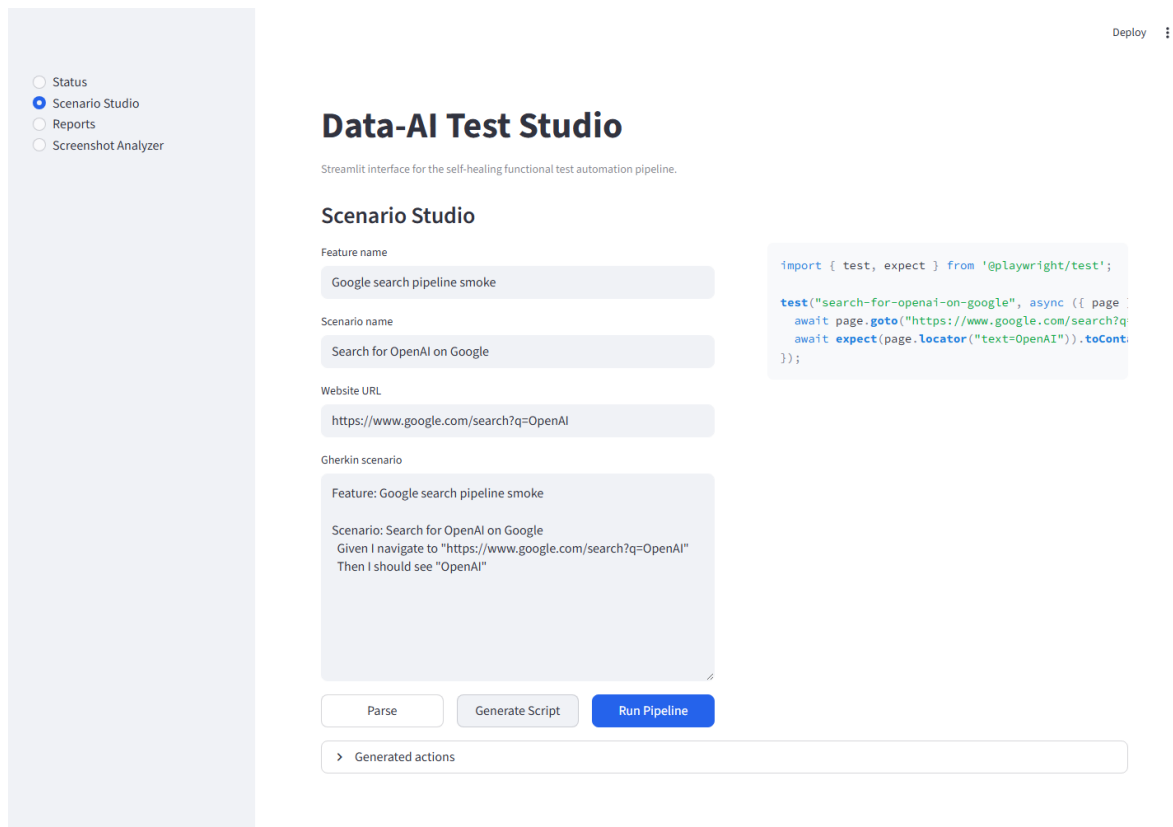


Figure 5.4: Generated Playwright script displayed in the Streamlit interface

5.5.3 Screenshot Analyzer

The screenshot analyzer allows the user to test the vision module independently. The user can upload a screenshot, activate OCR, define a target text, and view detected elements.

This page was useful during development because it allowed isolated testing of the YOLO/OCR pipeline.

5.5.4 Reports Page

The reports page allows the user to browse generated reports and inspect previous executions. It provides access to both HTML and JSON versions of each report.

Figure 5.5 shows the reports page used to inspect previous executions.

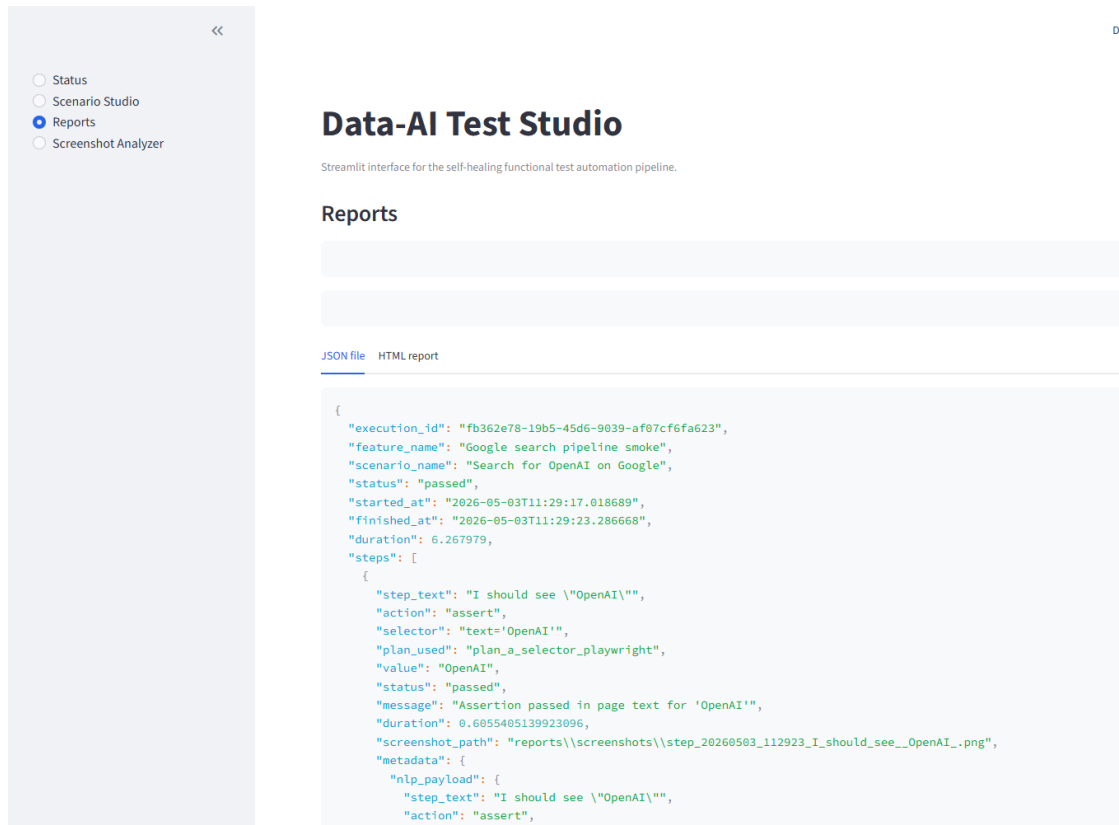


Figure 5.5: Reports page in the Streamlit interface

5.6 Deployment Attempts

Deployment was an important part of the project because the system includes several components: a backend API, browser automation, AI models, OCR dependencies, and a Streamlit frontend.

5.6.1 Local Deployment

The first stable deployment was local. The FastAPI backend was launched locally, and the Streamlit interface communicated with it through API calls.

This environment was useful for development and testing because it allowed direct access to logs, screenshots, reports, and model files.

5.6.2 Docker Deployment

Docker [7] was considered to make the application easier to run in different environments. Containerization helps package the backend, dependencies, and configuration in a reproducible way.

However, Docker deployment required careful handling of Playwright browsers, OCR dependencies, model files, and report storage. These requirements made deployment more complex than a simple web application.

5.6.3 First Streamlit Cloud Attempt

A first cloud deployment attempt was made using Streamlit Cloud because it is simple and convenient for Python interfaces. However, this approach showed limitations for the full system.

Streamlit Cloud is well suited for lightweight data applications, but **VisionTest-AI** requires browser automation, AI model loading, OCR, and backend services. Running all these components in the same lightweight environment created resource and configuration problems.

5.6.4 Resource Limit Issue

The main issue was resource limitation. Playwright requires browser binaries, YOLO requires model loading, and OCR may require system-level dependencies. These elements can exceed the constraints of a simple Streamlit deployment.

This experience showed that the frontend and backend should not be deployed in the same environment.

5.6.5 Final Proposed Architecture: Railway Backend and Streamlit UI

The final proposed deployment architecture separates the backend from the frontend:

- **FastAPI backend:** Deployed on a platform such as Railway [17].
- **Streamlit frontend:** Deployed separately and configured to call the backend API.
- **Reports and screenshots:** Stored by the backend service.
- **AI models:** Loaded by the backend where more resources are available.

This architecture is more realistic because it separates user interaction from heavy processing.

Figure 5.6 represents the final proposed deployment architecture. The diagram was created by the author using Mermaid [12].

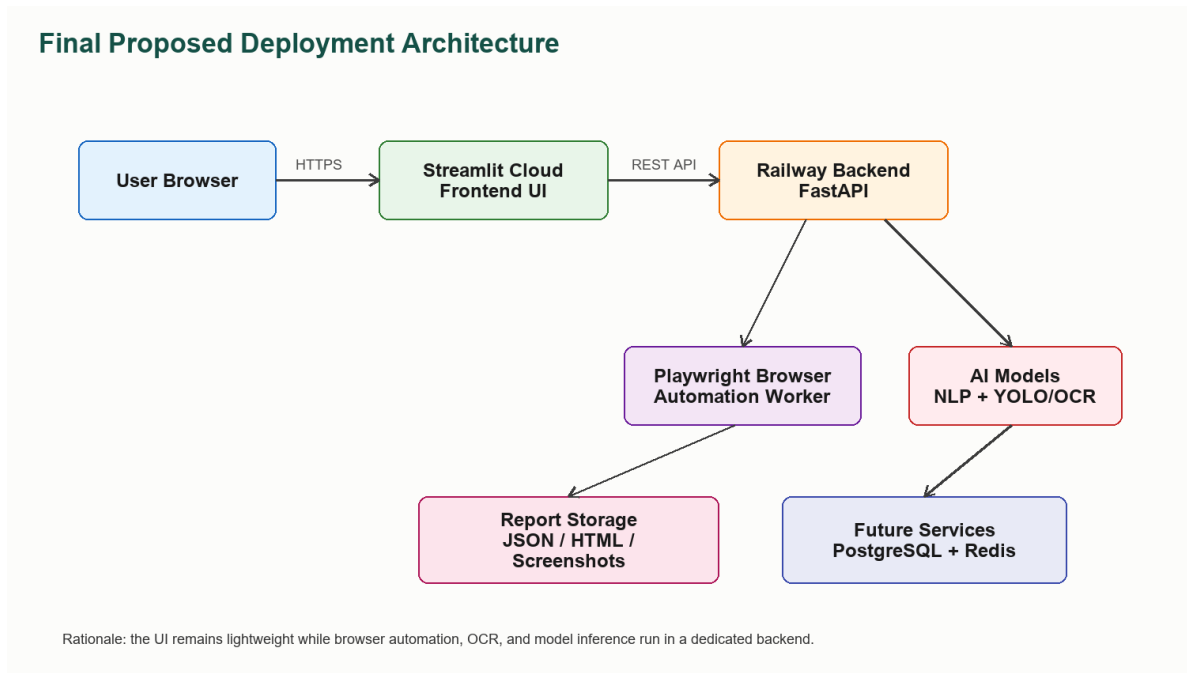


Figure 5.6: Final proposed deployment architecture

5.6.6 CI/CD Integration Strategy

Although a complete production CI/CD pipeline was not fully deployed during the internship, the project was structured to support this evolution. The objective of the CI/CD strategy is to automate verification before each deployment and reduce the risk of breaking the testing platform itself.

The proposed CI/CD workflow is composed of the following stages:

1. **Source code update:** A developer pushes changes to the project repository.
2. **Static verification:** The pipeline checks formatting, dependencies, and basic import errors.
3. **Unit and API tests:** Tests are executed for the mapping service, report service, generation service, and API endpoints.
4. **Docker build:** The backend image is built with the required Python dependencies, Playwright configuration, OCR tools, and model files.
5. **Deployment:** If tests pass, the backend can be deployed to Railway and the Streamlit frontend can be deployed separately.
6. **Post-deployment smoke test:** A simple scenario is executed to verify that the API, browser automation, and report generation still work after deployment.

In the current version, the project already contains automated tests and Docker configuration, which represent the foundation of this CI/CD strategy. Future work should connect these steps to a Git-based pipeline in order to automate deployment and validation.

5.7 Testing and Validation

The system was validated using several types of tests. The objective was to verify individual modules and the complete pipeline.

5.7.1 Unit Tests

Unit tests were used to validate isolated services such as the mapping service, report service, generator service, and utility functions.

These tests help ensure that the internal logic remains stable when changes are made.

5.7.2 API Tests

API tests were used to verify that the main endpoints return correct responses. These tests covered endpoints such as semantic mapping, test generation, segmentation, and reporting.

Example tested endpoints include:

- `/api/ia/semantic-mapping`
- `/api/ia/generate-test`
- `/api/ia/reports`
- `/api/ia/reports/{execution_id}/summary`

5.7.3 End-to-End Test Scenarios

End-to-end scenarios were used to test the complete pipeline. These scenarios started from Gherkin input and ended with generated reports.

The tested workflows included:

- Google search scenario.
- Facebook login scenario.
- Form filling scenario.
- Verification of expected text.

5.7.4 Unseen-Site Benchmark

The system was also tested on websites not used during development. This type of testing is important because it checks whether the pipeline can generalize beyond known examples.

The results were mixed. The NLP and mapping modules performed well on explicit steps, while the vision module still depended strongly on the quality of detections and OCR.

5.8 Actual Results

This section presents the main results obtained from the final prototype. The goal is to describe what works, what partially works, and what still needs improvement.

5.8.1 Generated JSON Reports

The system successfully generated JSON reports after execution. These reports contain structured information about the scenario, steps, screenshots, and execution summary.

Figure 5.7 presents an example of a generated JSON report.

Generated JSON Report - report_9602390c-ff7e-4b5b-9acc-79268fa0200a_20260513_161146.json

```
{
  "execution_id": "9602390c-ff7e-4b5b-9acc-79268fa0200a",
  "feature_name": "Google search pipeline smoke",
  "scenario_name": "Search for OpenAI on Google",
  "status": "passed",
  "summary": {
    "total_steps": 1,
    "passed_steps": 1,
    "failed_steps": 0,
    "duration": 16.266566,
    "plan_a_steps": 1,
    "plan_b_steps": 0
  },
  "first_step": {
    "step_text": "I should see \"OpenAI\"",
    "action": "assert",
    "selector": "text='OpenAI'",
    "plan_used": "plan_a_selector_playwright",
    "value": "OpenAI",
    "status": "passed",
    "message": "Assertion passed in page text for 'OpenAI'",
    "duration": 0.7042503356933594,
    "screenshot_path": "reports\\screenshots\\step_20260513_161145_I_should_see__OpenAI_.png",
    "metadata": {
      "nlp_payload": {
        "step_text": "I should see \"OpenAI\"",
        "action": "assert",
        "target": "text='OpenAI'",
        "value": "OpenAI",
        "selector": "text='OpenAI'",
        "intent": "VERIFICATION",
        "metadata": {
          "raw_nlp": {
            "step": "I should see \"OpenAI\"",
            "intent": "VERIFICATION",
            "values": [
              "OpenAI"
            ],
            "target": "OpenAI",
            "element_type": "element"
          }
        }
      }
    }
  }
}
```

Figure 5.7: Example of a generated JSON report

5.8.2 Generated HTML Reports

The system also generated HTML reports. These reports provide a more readable representation of the execution and are easier to analyze manually.

Figure 5.8 represents the final HTML report generated by the reporting module.

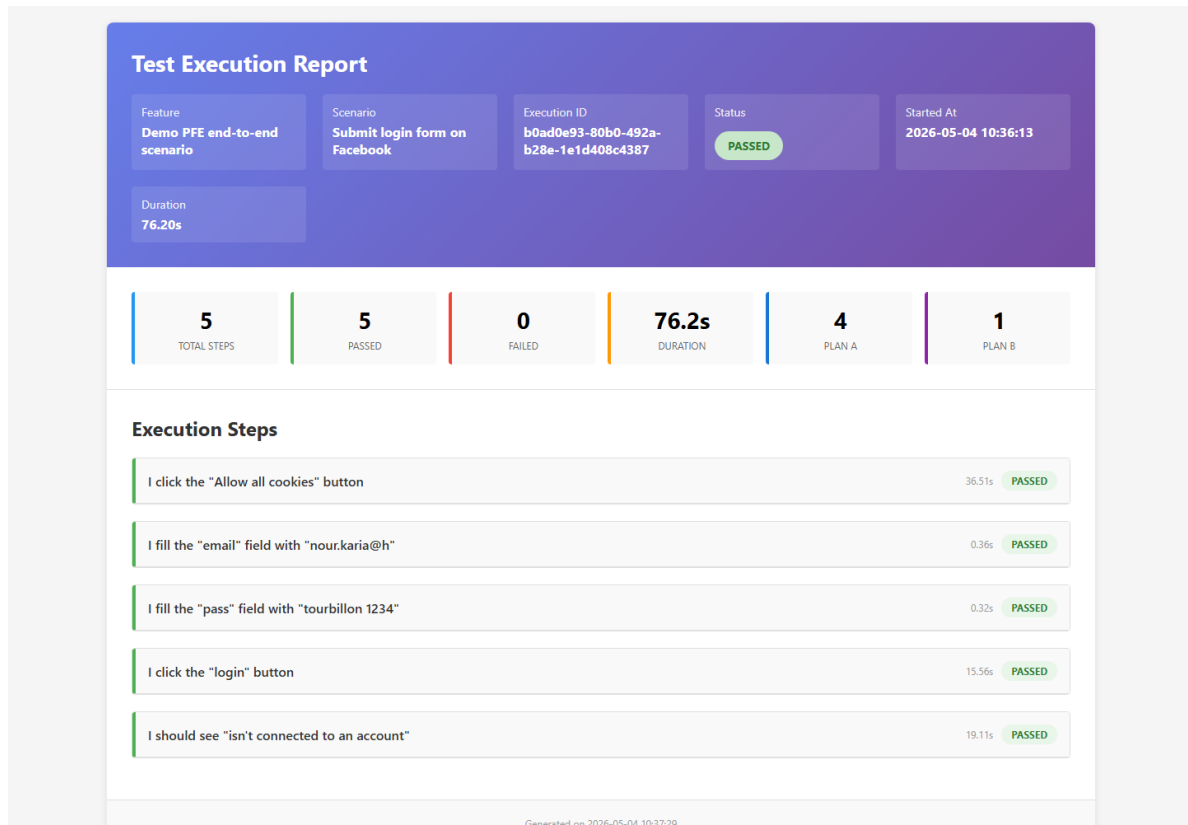


Figure 5.8: Example of a generated HTML report

5.8.3 Successful Execution Examples

Some scenarios were executed successfully from beginning to end. The most stable cases were scenarios with explicit steps and clear UI targets.

Examples include:

- Navigating to a target page.
- Filling a visible input field.
- Clicking a clearly identified button.
- Verifying that expected text appears on the page.

5.8.4 Failed Execution Examples

Some scenarios failed due to selector issues, page changes, popups, OCR errors, or ambiguous targets. These failures were useful because they helped identify the current limits of the

system.

Table 5.3 lists representative failed execution cases and possible improvements.

Table 5.3: Examples of failed execution cases

Failure Case	Cause	Possible Improvement
Button not clicked	Selector failed and visual detection was not precise enough.	Improve YOLO dataset and candidate ranking.
Wrong element selected	Several visual candidates were similar.	Combine OCR, DOM context, and semantic scoring.
OCR text incorrect	Text was small or low contrast.	Improve preprocessing and OCR configuration.
Cloud execution issue	Deployment environment lacked required resources.	Separate backend and front-end deployment.

5.8.5 Plan A vs Plan B Usage

The execution strategy is divided into Plan A and Plan B. Plan A uses Playwright selectors, while Plan B uses visual fallback.

In the current version, Plan A remains the most reliable strategy when selectors are valid. Plan B is useful in some cases, but its success depends on YOLO detection quality, OCR readability, and target ranking.

Table 5.4 compares the two execution strategies used by the system.

Table 5.4: Comparison between Plan A and Plan B

Strategy	Strengths	Limitations
Plan A: Playwright selectors	Fast, precise, reliable when DOM is stable.	Breaks when selectors change or elements are hard to locate.
Plan B: YOLO/OCR fallback	Can recover when the element is visible but selectors fail.	Depends on model accuracy, OCR quality, and ranking logic.

5.8.6 Performance Observations

The NLP and semantic mapping modules are lightweight and respond quickly for simple scenarios. Playwright execution time depends mainly on page loading, network speed, and browser behavior.

The vision module is more expensive because it requires screenshot processing, YOLO inference, and OCR. For this reason, it is only activated when needed, rather than being used for every action.

This design keeps the normal execution path efficient while still allowing fallback when necessary.

5.9 Limitations

The final prototype achieved the main objective of connecting Gherkin understanding, test generation, execution, vision fallback, and reporting. However, several limitations remain.

5.9.1 Model Accuracy Limitations

The YOLO model still requires improvement. Some detections are inaccurate, especially for small elements, dense interfaces, or visually similar components. The dataset must be expanded and cleaned further to improve model generalization.

5.9.2 OCR Limitations

OCR performance depends on image quality, font size, contrast, and page layout. In some cases, OCR extracted incorrect or incomplete text, which affected target matching.

5.9.3 Deployment Resource Limitations

The deployment phase showed that the system requires more resources than a simple Streamlit application. Browser automation, OCR, and AI model inference should be handled by a dedicated backend service.

5.9.4 Lack of PostgreSQL and Redis Integration

The current version stores reports as local JSON and HTML files. This is sufficient for a prototype, but not ideal for a production system.

A future version should use:

- **PostgreSQL** for persistent storage of projects, executions, users, and reports.
- **Redis** for caching, queues, and background execution management.

5.10 Perspectives

Several improvements can be added in future versions of **VisionTest-AI**.

5.10.1 Improve NLP Multilingual Support

The current NLP module mainly supports English scenarios. Future work should add support for French and mixed-language Gherkin scenarios.

5.10.2 Expand the Annotated Dataset

The vision dataset should be expanded with more websites, component libraries, screen sizes, themes, and UI styles. This would improve YOLO detection quality and reduce false positives.

5.10.3 Add PostgreSQL and Redis

A production version should include database and queue services. PostgreSQL can store reports and project history, while Redis can manage background tasks and improve performance.

5.10.4 Add Authentication and Project History

Authentication would allow multiple users to manage their own projects, scenarios, and reports. Project history would make it possible to compare executions over time.

5.10.5 Improve Cloud Deployment

The final deployment architecture should separate the frontend, backend, browser execution, model inference, and report storage. This would make the system more stable and scalable.

5.11 Conclusion

This chapter presented the final integration of **VisionTest-AI**. It showed how the different modules are connected to form a complete pipeline, starting from a Gherkin scenario and ending with JSON and HTML execution reports.

The backend implementation was organized around FastAPI routers and services. The Streamlit interface provided access to scenario submission, script generation, screenshot analysis, and report viewing. The reporting system made it possible to store and inspect execution evidence.

The deployment attempts showed that the system is more complex than a simple web application because it includes browser automation, AI models, OCR, screenshots, and reports. This led to the final proposed architecture based on a separated backend and frontend.

The final results confirm that the prototype can parse scenarios, generate Playwright scripts, execute tests, produce reports, and activate visual fallback in some cases. However, the project still has limitations related to YOLO accuracy, OCR reliability, deployment resources, and missing production features such as PostgreSQL, Redis, authentication, and project history.

Overall, **VisionTest-AI** demonstrates the feasibility of an intelligent functional testing agent that combines semantic understanding, browser automation, visual perception, and reporting. The system provides a solid foundation for future improvements and industrialization.

General Conclusion and Perspectives

This graduation project focused on the design and implementation of **VisionTest-AI**, an intelligent functional testing agent that combines natural language understanding, browser automation, visual perception, and automatic reporting. The main objective was to reduce the fragility of traditional automated tests by allowing the system to understand Gherkin scenarios, generate executable actions, execute them with Playwright, and use visual fallback mechanisms when classical selectors fail.

Throughout the project, we built a complete prototype composed of several modules. The NLP module parses Gherkin scenarios, classifies intents, extracts entities, and prepares structured action payloads. The semantic mapping module transforms these actions into Playwright-compatible commands and selector candidates. The execution service runs the test steps in a browser, while the vision module uses YOLOv8n and OCR to detect UI elements from screenshots when selector-based execution is not sufficient. Finally, the reporting service generates JSON and HTML reports containing execution details, screenshots, errors, and summary metrics.

One of the main achievements of this work is the integration of several technologies into a single functional pipeline. The project does not only generate scripts or analyze screenshots separately; it connects these components into a complete workflow, starting from a business-readable Gherkin scenario and ending with an execution report. This demonstrates the feasibility of an intelligent testing assistant that can support QA engineers in writing, executing, and analyzing functional tests.

The project also highlighted several technical challenges. Preparing a reliable UI detection dataset was more difficult than expected. Ready-made datasets were not diverse enough, manual annotation was time-consuming, and automatic labeling required several corrections before becoming usable. The final DOM-assisted labeling approach helped improve the quality of annotations, but the YOLO model still needs more training data and refinement. Deployment was another challenge, especially because the system requires browser automation, OCR, AI models, and report storage, which makes it heavier than a simple Streamlit application.

The obtained results are encouraging. The NLP and semantic mapping modules successfully handled explicit functional steps such as navigation, input, click, and verification. The system was able to generate Playwright scripts, execute scenarios, capture screenshots, and produce structured JSON and HTML reports. The visual fallback mechanism also showed its value in some cases, especially when the target element was visible but the selector-based

strategy failed.

However, the current version remains a prototype and has several limitations. The vision module is not yet fully reliable on all interfaces, OCR can fail on small or low-contrast text, and some complex interactions such as drag-and-drop, file upload, or advanced dynamic components are not fully supported. The system also does not yet include production features such as authentication, project history, PostgreSQL storage, Redis queues, or advanced cloud orchestration.

Several perspectives can be considered for future work. The NLP module can be improved by adding multilingual support, especially for French and mixed-language Gherkin scenarios. The vision dataset can be expanded with more annotated examples from different websites, themes, and component libraries. The execution engine can be enhanced with better ranking logic, stronger self-healing strategies, and support for more complex user actions. On the platform side, adding PostgreSQL, Redis, authentication, and project management features would make the system closer to an industrial solution.

In conclusion, **VisionTest-AI** provides a solid foundation for an AI-assisted functional testing platform. It shows that combining semantic understanding with visual perception can make test automation more flexible and more resilient. Although further improvements are still required, the project successfully demonstrates how AI techniques can be applied to modern QA workflows in a practical and extensible way.

Bibliography

- [1] Applitools. Applitools visual ai testing platform. <https://applitools.com/>, 2026. Accessed: 2026-05-15.
- [2] Atlassian. Trello. <https://trello.com/>, 2026. Accessed: 2026-05-15.
- [3] Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [4] Cucumber. Gherkin reference. <https://cucumber.io/docs/gherkin/reference/>, 2026. Accessed: 2026-05-15.
- [5] Cypress. Cypress documentation. <https://docs.cypress.io/>, 2026. Accessed: 2026-05-15.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [7] Docker. Docker documentation. <https://docs.docker.com/>, 2026. Accessed: 2026-05-15.
- [8] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Accessed: 2026-05-15.
- [9] Functionize. Functionize ai-powered testing platform. <https://www.functionize.com/>, 2026. Accessed: 2026-05-15.
- [10] Git. Git documentation. <https://git-scm.com/doc>, 2026. Accessed: 2026-05-15.
- [11] Mabl. Mabl ai-native test automation. <https://www.mabl.com/>, 2026. Accessed: 2026-05-15.
- [12] Mermaid. Mermaid documentation. <https://mermaid.js.org/>, 2026. Accessed: 2026-05-16.
- [13] Microsoft. Playwright documentation. <https://playwright.dev/>, 2026. Accessed: 2026-05-15.
- [14] Microsoft. Visual studio code documentation. <https://code.visualstudio.com/docs>, 2026. Accessed: 2026-05-15.

- [15] Pydantic. Pydantic documentation. <https://docs.pydantic.dev/>, 2026. Accessed: 2026-05-15.
- [16] QA Wolf. Qa wolf end-to-end test automation. <https://www.qawolf.com/>, 2026. Accessed: 2026-05-15.
- [17] Railway. Railway documentation. <https://docs.railway.com/>, 2026. Accessed: 2026-05-15.
- [18] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *arXiv preprint arXiv:1506.02640*, 2015.
- [19] Roboflow. Roboflow documentation. <https://docs.roboflow.com/>, 2026. Accessed: 2026-05-15.
- [20] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [21] Selenium Project. Selenium documentation. <https://www.selenium.dev/documentation/>, 2026. Accessed: 2026-05-15.
- [22] Ray Smith. An overview of the tesseract ocr engine. In *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, pages 629–633, 2007.
- [23] Streamlit. Streamlit documentation. <https://docs.streamlit.io/>, 2026. Accessed: 2026-05-15.
- [24] Tesseract OCR. Tesseract ocr documentation. <https://tesseract-ocr.github.io/>, 2026. Accessed: 2026-05-15.
- [25] Tricentis. Testim test automation platform. <https://www.tricentis.com/products/testim>, 2026. Accessed: 2026-05-15.
- [26] Tzutalin. Labelimg. <https://github.com/HumanSignal/labelImg>, 2026. Accessed: 2026-05-15.
- [27] Ultralytics. YOLOv8 documentation. <https://docs.ultralytics.com/models/yolov8/>, 2026. Accessed: 2026-05-15.
- [28] Matt Wynne, Aslak Hellesoy, and Steve Tooke. *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf, 2nd edition, 2017.